

Business Case LoanTap Logistic Regression

LoanTap is an online platform committed to delivering customized loan products to millennials. They innovate in an otherwise dull loan segment, to deliver instant, flexible loans on consumer friendly terms to salaried professionals and businessmen.

The data science team at LoanTap is building an underwriting layer to determine the creditworthiness of MSMEs as well as individuals.

LoanTap deploys formal credit to salaried individuals and businesses 4 main financial instruments:

- Personal Loan
- EMI Free Loan
- Personal Overdraft
- Advance Salary Loan This case study will focus on the underwriting process behind Personal Loan only

Problem Statement:

Given a set of attributes for an Individual, determine if a credit line should be extended to them. If so, what should the repayment terms be in business recommendations? LoanTap wants to:

- Predict whether a borrower will default
- Reduce Non-Performing Assets (NPAs)
- Optimize approval strategy
- Enable risk-based pricing

```
# Basic libraries
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing & modeling
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Evaluation
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import roc_auc_score
```

```

from sklearn.metrics import roc_curve
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import auc

```

```

# Ignore warnings

```

```

import warnings
warnings.filterwarnings('ignore')

```

```

df = pd.read_csv("logistic_regression.csv")
df.head(5)

```

	loan_amnt	term	int_rate	installment	grade	sub_grade	\
0	10000.0	36 months	11.44	329.48	B	B4	
1	8000.0	36 months	11.99	265.68	B	B5	
2	15600.0	36 months	10.49	506.97	B	B3	
3	7200.0	36 months	6.49	220.65	A	A2	
4	24375.0	60 months	17.27	609.33	C	C5	

	emp_title	emp_length	home_ownership	annual_inc	\
0	Marketing	10+ years	RENT	117000.0	
1	Credit analyst	4 years	MORTGAGE	65000.0	
2	Statistician	< 1 year	RENT	43057.0	
3	Client Advocate	6 years	RENT	54000.0	
4	Destiny Management Inc.	9 years	MORTGAGE	55000.0	

	verification_status	issue_d	loan_status	purpose	\
0	Not Verified	Jan-2015	Fully Paid	vacation	
1	Not Verified	Jan-2015	Fully Paid	debt_consolidation	
2	Source Verified	Jan-2015	Fully Paid	credit_card	
3	Not Verified	Nov-2014	Fully Paid	credit_card	
4	Verified	Apr-2013	Charged Off	credit_card	

	title	dti	earliest_cr_line	open_acc	pub_rec	\
0	Vacation	26.24	Jun-1990	16.0	0.0	
1	Debt consolidation	22.05	Jul-2004	17.0	0.0	
2	Credit card refinancing	12.79	Aug-2007	13.0	0.0	
3	Credit card refinancing	2.60	Sep-2006	6.0	0.0	
4	Credit Card Refinance	33.95	Mar-1999	13.0	0.0	

	revol_bal	revol_util	total_acc	initial_list_status	application_type	\
0	36369.0	41.8	25.0		w	INDIVIDUAL
1	20131.0	53.3	27.0		f	INDIVIDUAL

2	11987.0	92.2	26.0	f
INDIVIDUAL				
3	5472.0	21.5	13.0	f
INDIVIDUAL				
4	24584.0	69.8	43.0	f
INDIVIDUAL				

	mort_acc	pub_rec_bankruptcies	\
0	0.0	0.0	
1	3.0	0.0	
2	0.0	0.0	
3	0.0	0.0	
4	1.0	0.0	

	address
0	0174 Michelle Gateway\r\nMendozaberg, OK 22690
1	1076 Carney Fort Apt. 347\r\nLoganmouth, SD 05113
2	87025 Mark Dale Apt. 269\r\nNew Sabrina, WV 05113
3	823 Reid Ford\r\nDelacruzside, MA 00813
4	679 Luna Roads\r\nGreggshire, VA 11650

Data dictionary:

- loan_amnt : The listed amount of the loan applied for by the borrower. If at some point in time, the credit department reduces the loan amount, then it will be reflected in this value.
- term : The number of payments on the loan. Values are in months and can be either 36 or 60.
- int_rate : Interest Rate on the loan
- installment : The monthly payment owed by the borrower if the loan originates.
- grade : LoanTap assigned loan grade
- sub_grade : LoanTap assigned loan subgrade
- emp_title : The job title supplied by the Borrower when applying for the loan.*
- emp_length : Employment length in years. Possible values are between 0 and 10 where 0 means less than one year and 10 means ten or more years.
- home_ownership : The home ownership status provided by the borrower during registration or obtained from the credit report.
- annual_inc : The self-reported annual income provided by the borrower during registration.
- verification_status : Indicates if income was verified by LoanTap, not verified, or if the income source was verified
- issue_d : The month which the loan was funded
- loan_status : Current status of the loan - Target Variable
- purpose : A category provided by the borrower for the loan request.
- title : The loan title provided by the borrower
- dti : A ratio calculated using the borrower's total monthly debt payments on the total debt obligations, excluding mortgage and the * requested LoanTap loan, divided by the borrower's self-reported monthly income.

- `earliest_cr_line` :The month the borrower's earliest reported credit line was opened
- `open_acc` : The number of open credit lines in the borrower's credit file.
- `pub_rec` : Number of derogatory public records
- `revol_bal` : Total credit revolving balance
- `revol_util` : Revolving line utilization rate, or the amount of credit the borrower is using relative to all available revolving credit.
- `total_acc` : The total number of credit lines currently in the borrower's credit file
- `initial_list_status` : The initial listing status of the loan. Possible values are – W, F
- `application_type` : Indicates whether the loan is an individual application or a joint application with two co-borrowers
- `mort_acc` : Number of mortgage accounts.
- `pub_rec_bankruptcies` : Number of public record bankruptcies
- `Address`: Address of the individual

```
print("Number of rows and columns in the dataset:")
df.shape
```

Number of rows and columns in the dataset:

(396030, 27)

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 396030 entries, 0 to 396029
```

```
Data columns (total 27 columns):
```

#	Column	Non-Null Count	Dtype
0	loan_amnt	396030 non-null	float64
1	term	396030 non-null	object
2	int_rate	396030 non-null	float64
3	installment	396030 non-null	float64
4	grade	396030 non-null	object
5	sub_grade	396030 non-null	object
6	emp_title	373103 non-null	object
7	emp_length	377729 non-null	object
8	home_ownership	396030 non-null	object
9	annual_inc	396030 non-null	float64
10	verification_status	396030 non-null	object
11	issue_d	396030 non-null	object
12	loan_status	396030 non-null	object
13	purpose	396030 non-null	object
14	title	394274 non-null	object
15	dti	396030 non-null	float64
16	earliest_cr_line	396030 non-null	object
17	open_acc	396030 non-null	float64
18	pub_rec	396030 non-null	float64
19	revol_bal	396030 non-null	float64
20	revol_util	395754 non-null	float64

```

21 total_acc          396030 non-null float64
22 initial_list_status 396030 non-null object
23 application_type     396030 non-null object
24 mort_acc            358235 non-null float64
25 pub_rec_bankruptcies 395495 non-null float64
26 address             396030 non-null object

```

dtypes: float64(12), object(15)

memory usage: 81.6+ MB

df.describe()

	loan_amnt	int_rate	installment	annual_inc \
count	396030.000000	396030.000000	396030.000000	3.960300e+05
mean	14113.888089	13.639400	431.849698	7.420318e+04
std	8357.441341	4.472157	250.727790	6.163762e+04
min	500.000000	5.320000	16.080000	0.000000e+00
25%	8000.000000	10.490000	250.330000	4.500000e+04
50%	12000.000000	13.330000	375.430000	6.400000e+04
75%	20000.000000	16.490000	567.300000	9.000000e+04
max	40000.000000	30.990000	1533.810000	8.706582e+06

	dti	open_acc	pub_rec	revol_bal \
count	396030.000000	396030.000000	396030.000000	3.960300e+05
mean	17.379514	11.311153	0.178191	1.584454e+04
std	18.019092	5.137649	0.530671	2.059184e+04
min	0.000000	0.000000	0.000000	0.000000e+00
25%	11.280000	8.000000	0.000000	6.025000e+03
50%	16.910000	10.000000	0.000000	1.118100e+04
75%	22.980000	14.000000	0.000000	1.962000e+04
max	9999.000000	90.000000	86.000000	1.743266e+06

	revol_util	total_acc	mort_acc
pub_rec_bankruptcies			
count	395754.000000	396030.000000	358235.000000
	395495.000000		
mean	53.791749	25.414744	1.813991
	0.121648		
std	24.452193	11.886991	2.147930
	0.356174		
min	0.000000	2.000000	0.000000
	0.000000		
25%	35.800000	17.000000	0.000000
	0.000000		
50%	54.800000	24.000000	1.000000
	0.000000		
75%	72.900000	32.000000	3.000000
	0.000000		
max	892.300000	151.000000	34.000000
	8.000000		

```
df.isnull().sum()
loan_amnt      0
term           0
int_rate       0
installment    0
grade          0
sub_grade      0
emp_title      22927
emp_length     18301
home_ownership 0
annual_inc     0
verification_status 0
issue_d        0
loan_status    0
purpose        0
title          1756
dti            0
earliest_cr_line 0
open_acc       0
pub_rec        0
revol_bal      0
revol_util     276
total_acc      0
initial_list_status 0
application_type 0
mort_acc       37795
pub_rec_bankruptcies 535
address         0
dtype: int64
```

▮ Observations

- 396,030 rows
- 27 columns
- 12 numeric, 15 categorical
- Missing values in:
 - emp_title
 - emp_length
 - mort_acc
 - pub_rec_bankruptcies

```
#TARGET VARIABLE TRANSFORMATION
#df['loan_status'] = df['loan_status'].apply(
#    lambda x: 1 if x in ['Charged Off', 'Default'] else 0
#)

#df['loan_status'].value_counts(normalize=True)
```

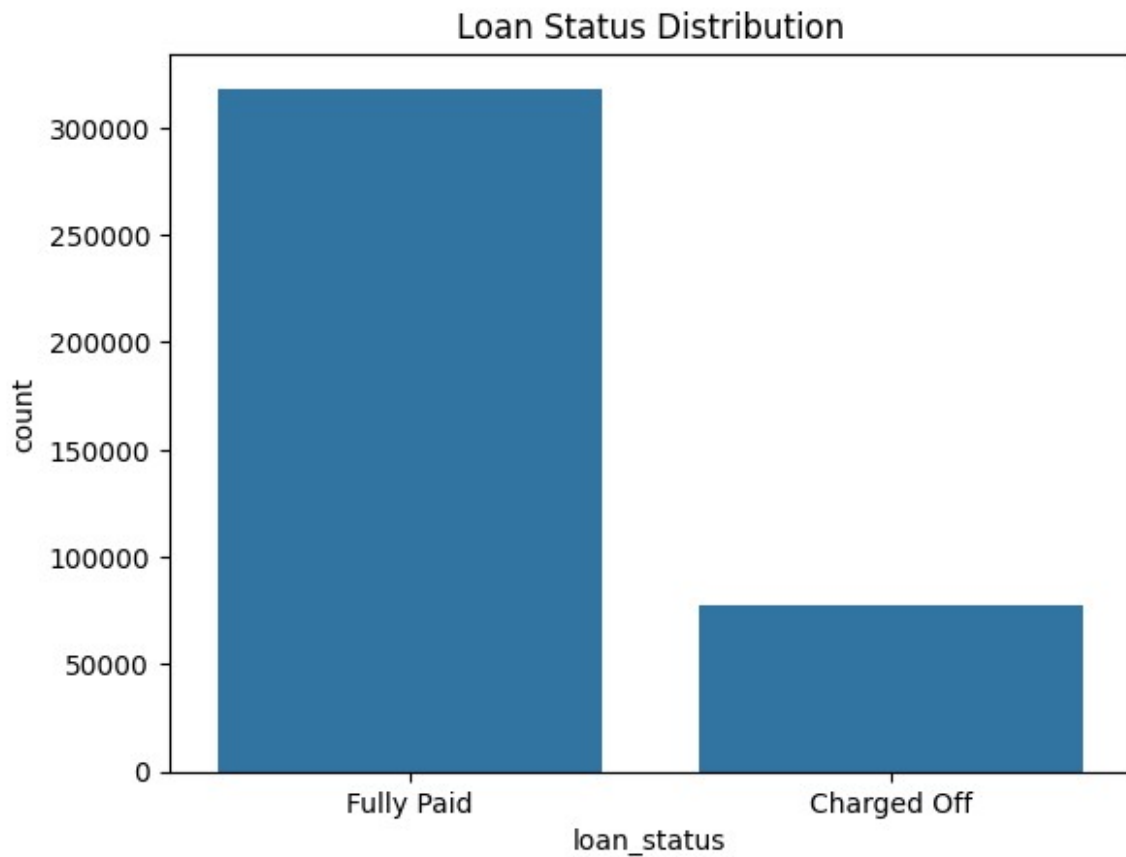
- Fully Paid (0) → 80.39%

- Default (1) → 19.61% Interpretation This is a moderately imbalanced classification problem (4:1 ratio). Therefore: Accuracy is misleading we need to focus on Recall (defaulters), ROC-AUC, PR curve

Exploratory Data Analysis (EDA)

Target Distribution

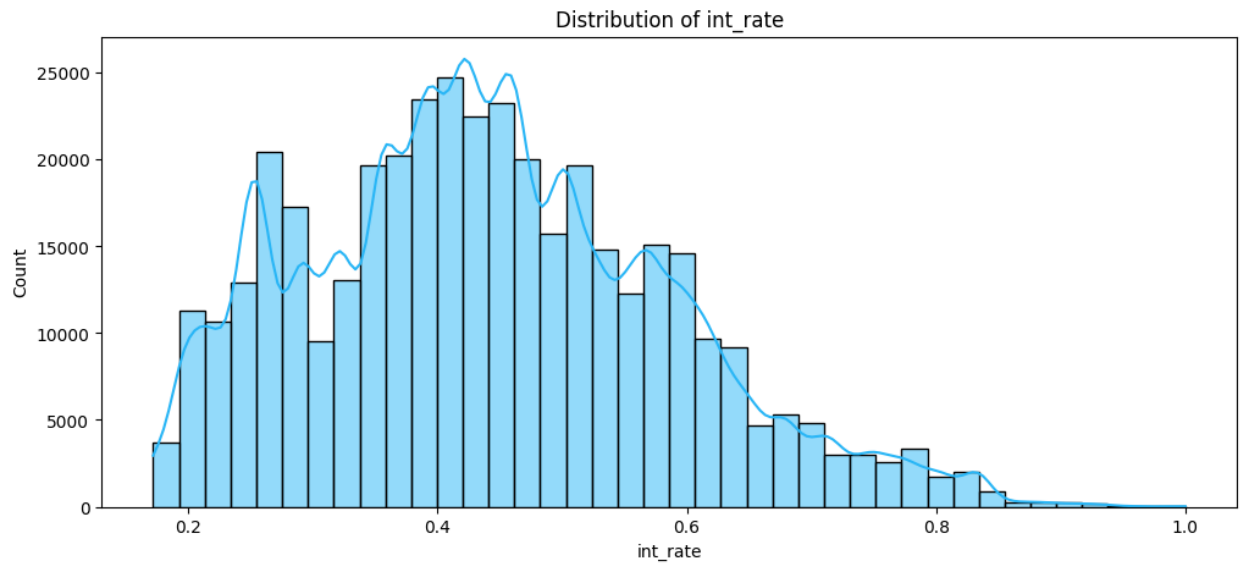
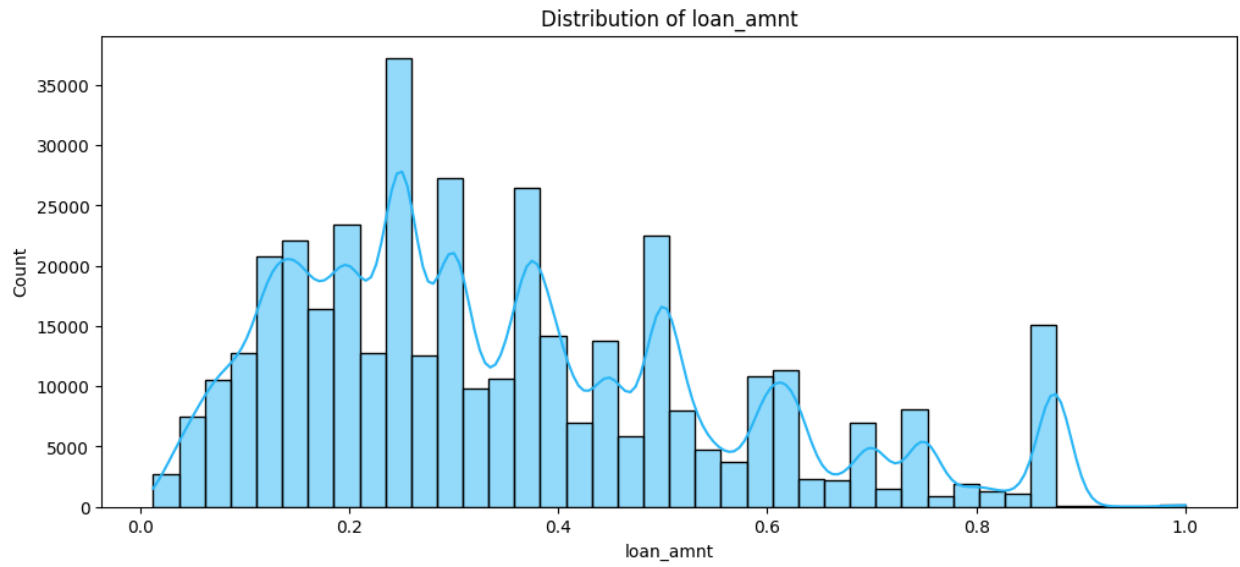
```
sns.countplot(x = "loan_status", data = df)
plt.title("Loan Status Distribution")
plt.show()
```

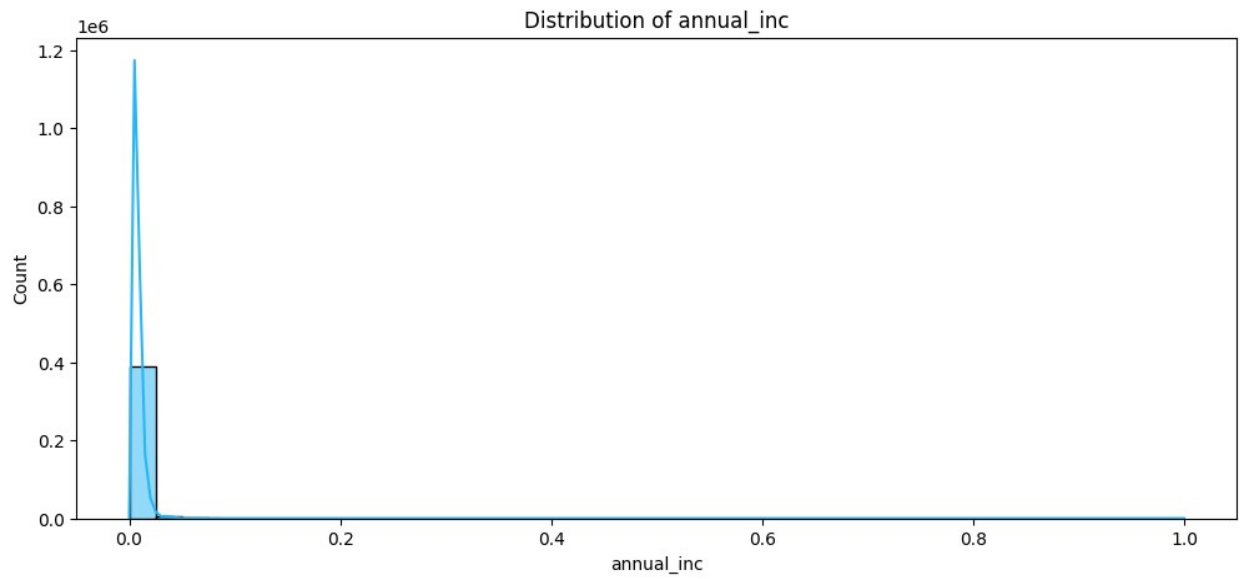
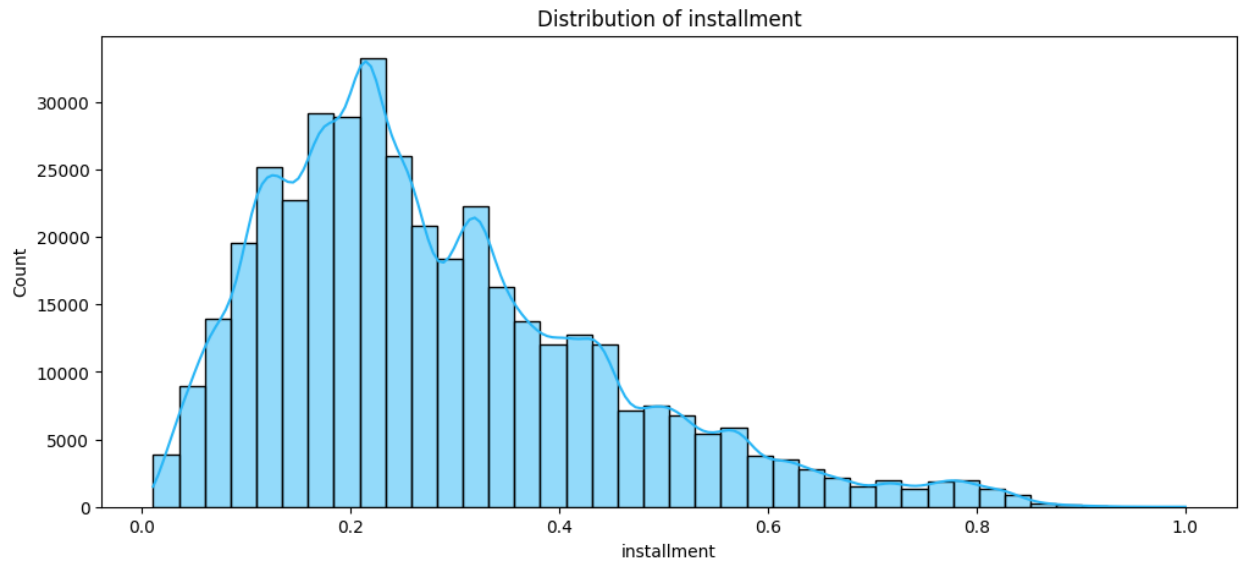


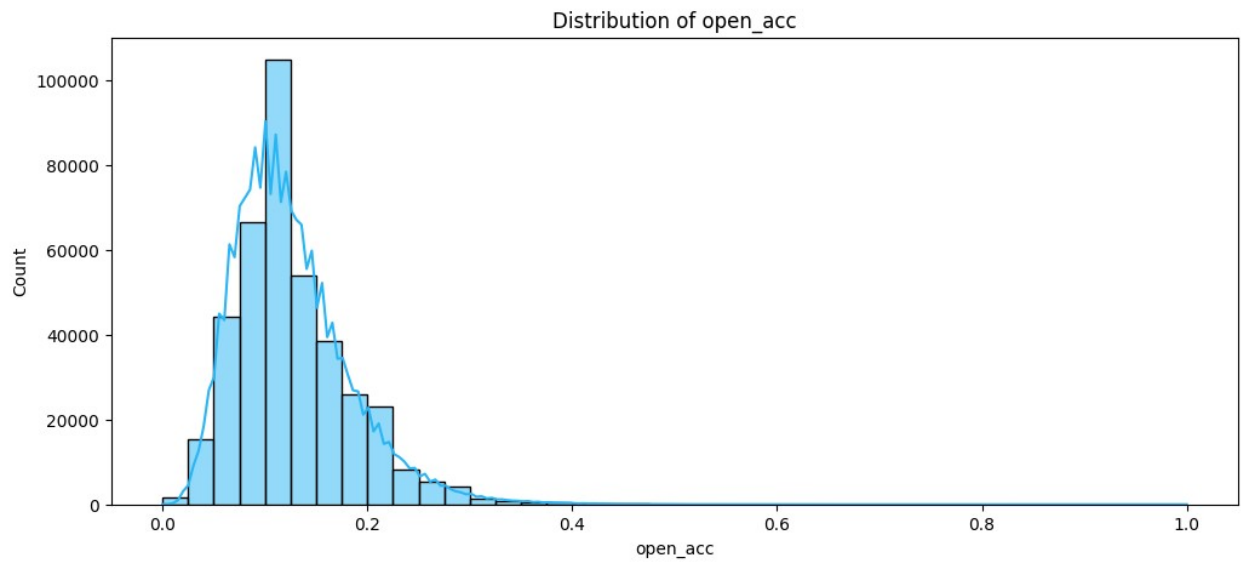
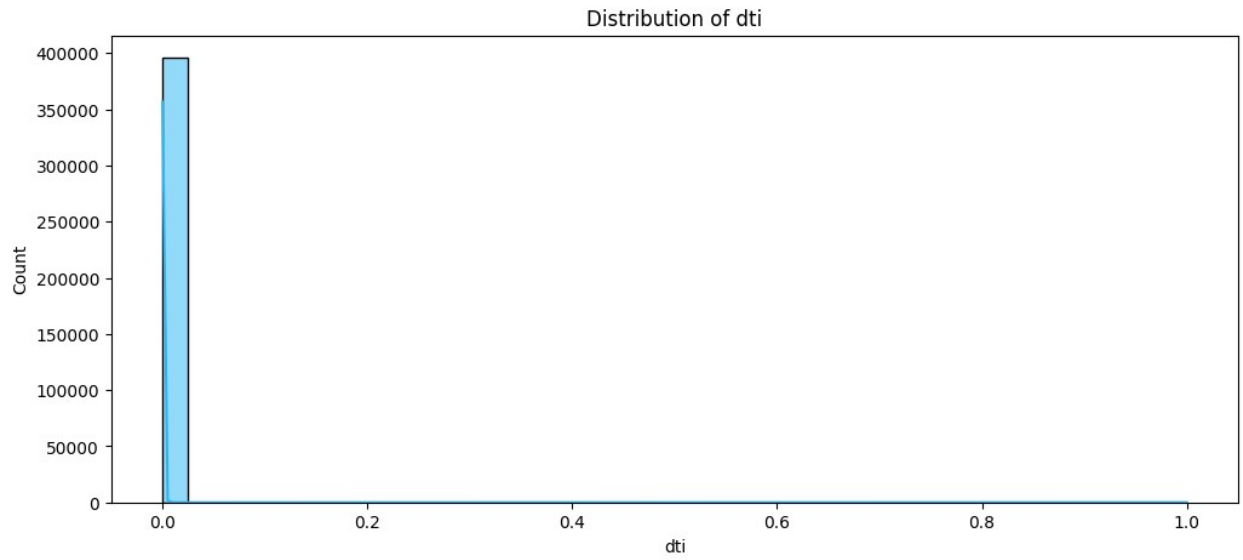
- Majority Fully Paid
- Default rate ~20% that implies Significant credit risk

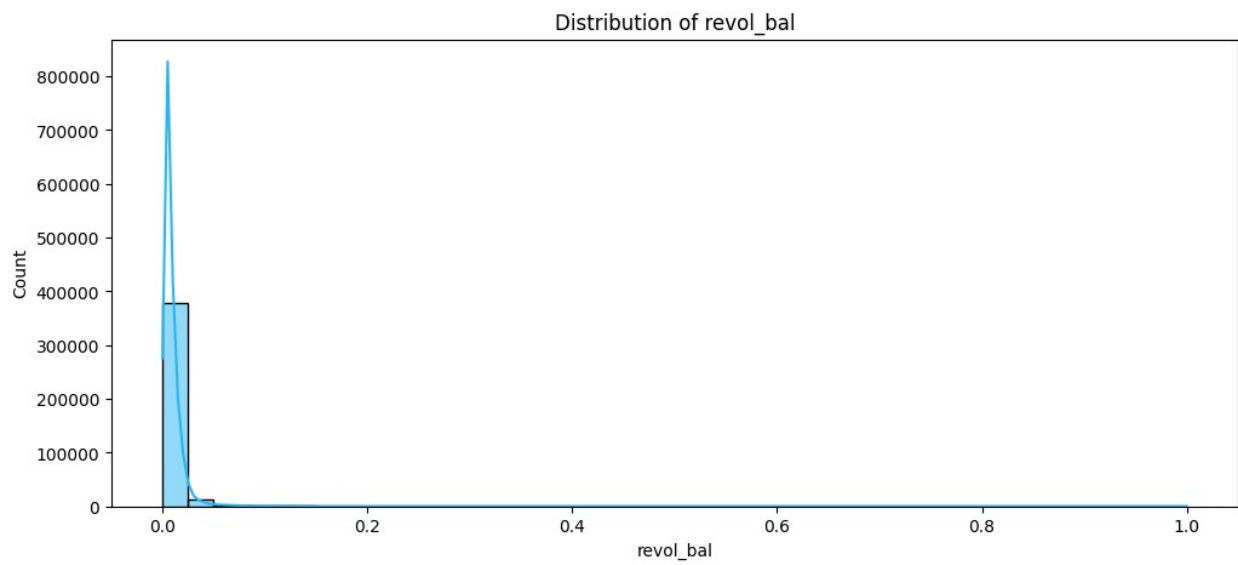
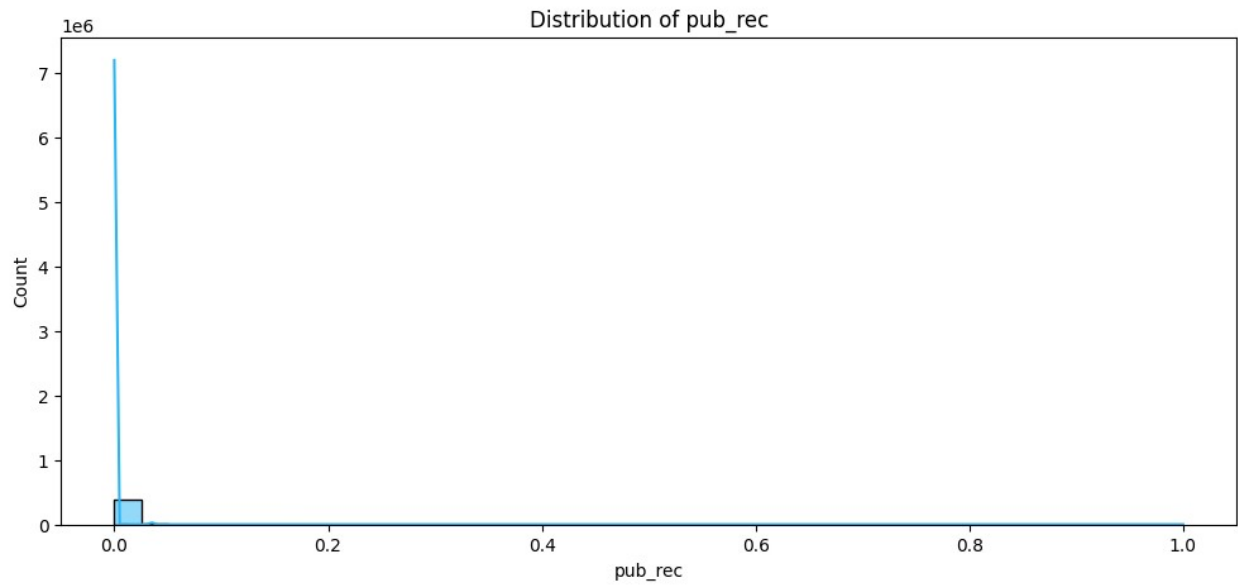
#Continuous Variables

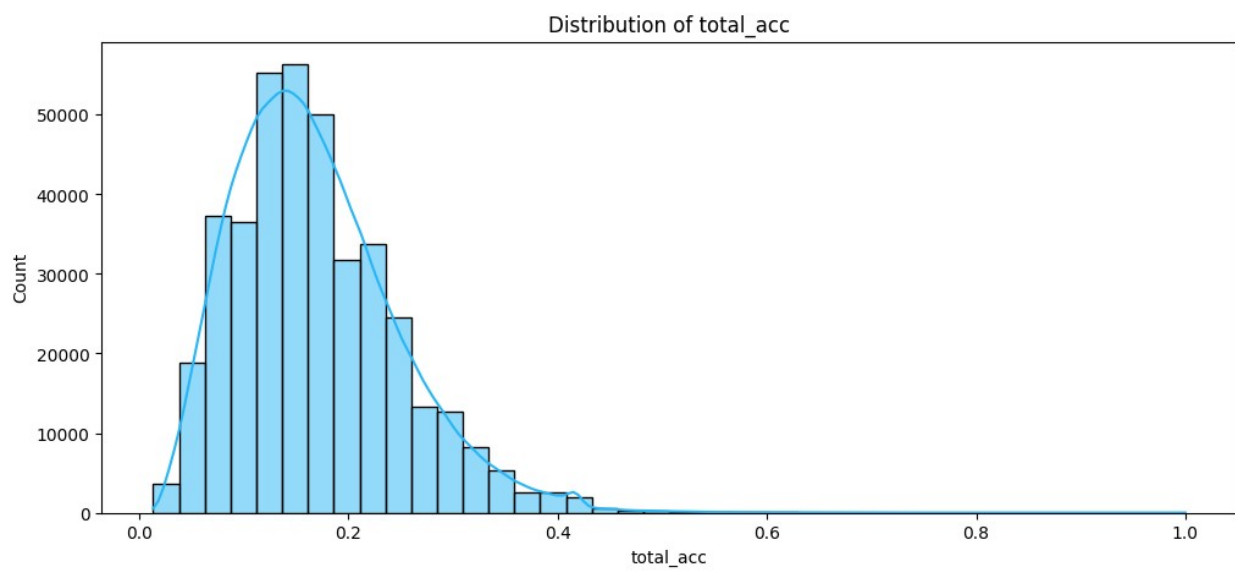
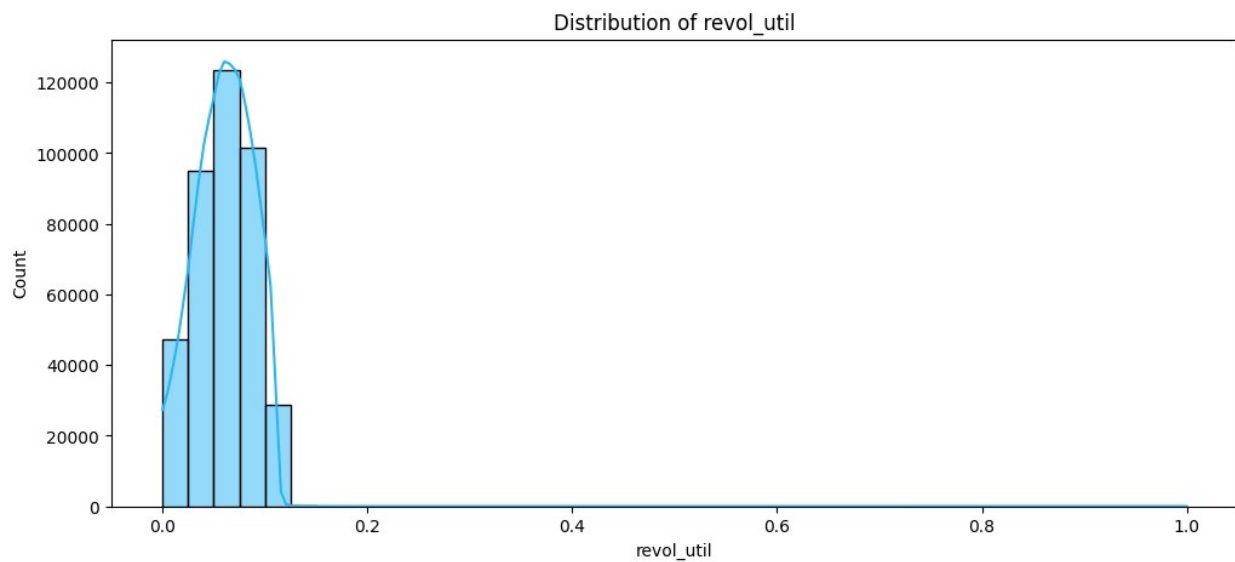
```
num_cols = df.select_dtypes('float64').columns.tolist()
for i in num_cols:
    plt.figure(figsize=(12,5))
    plt.title("Distribution of {}".format(i))
    sns.histplot(df[i]/df[i].max(), kde=True,color="#29B6F6", bins=40)
    plt.show()
```

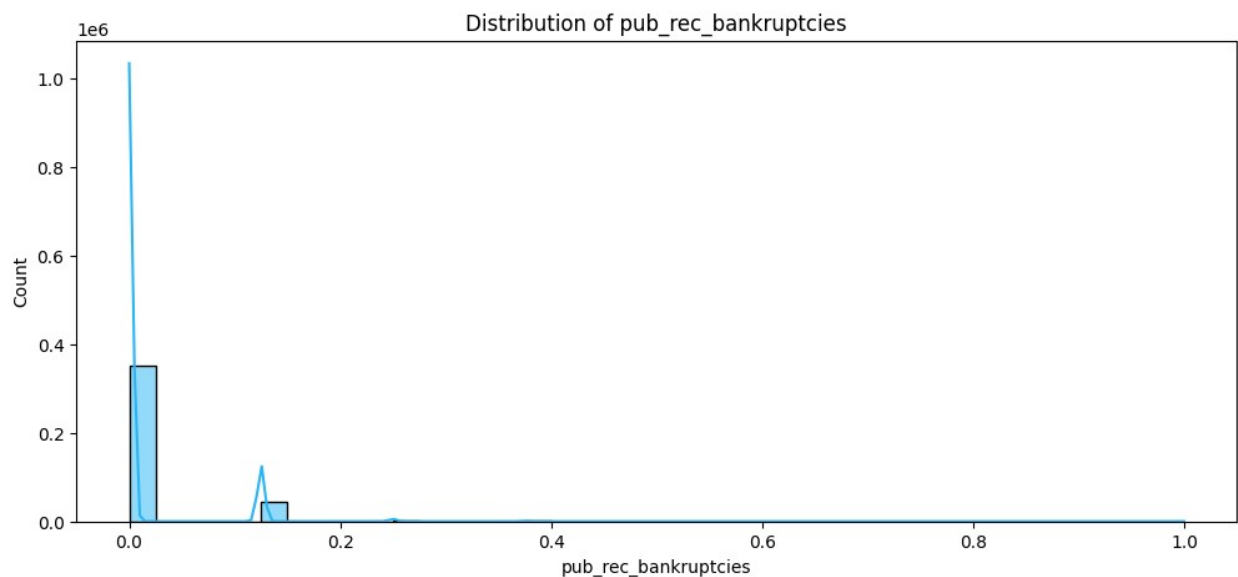
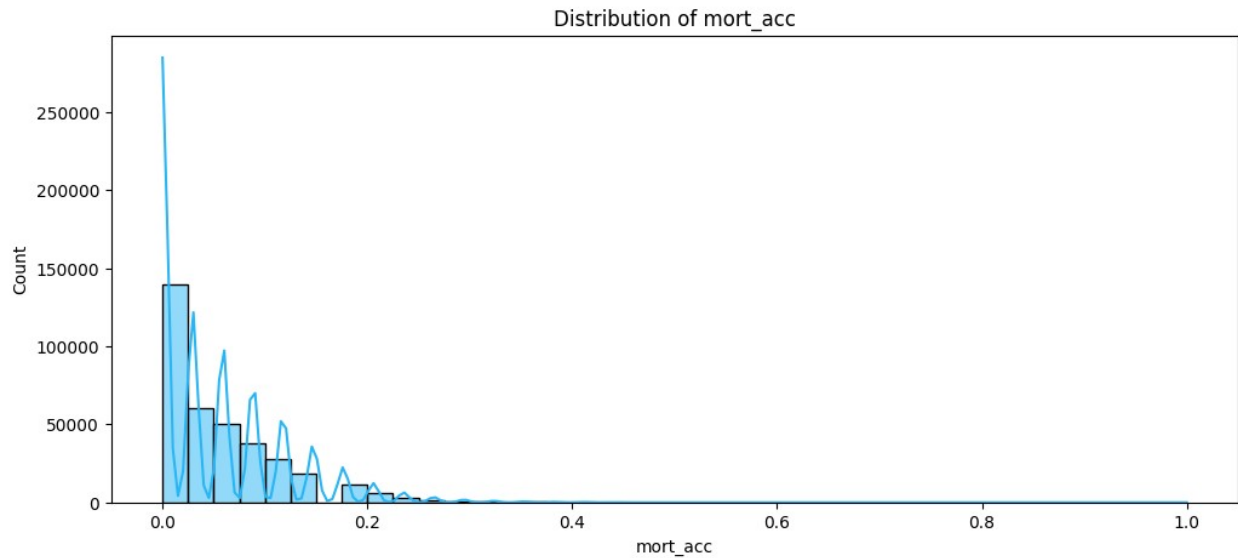












Observations

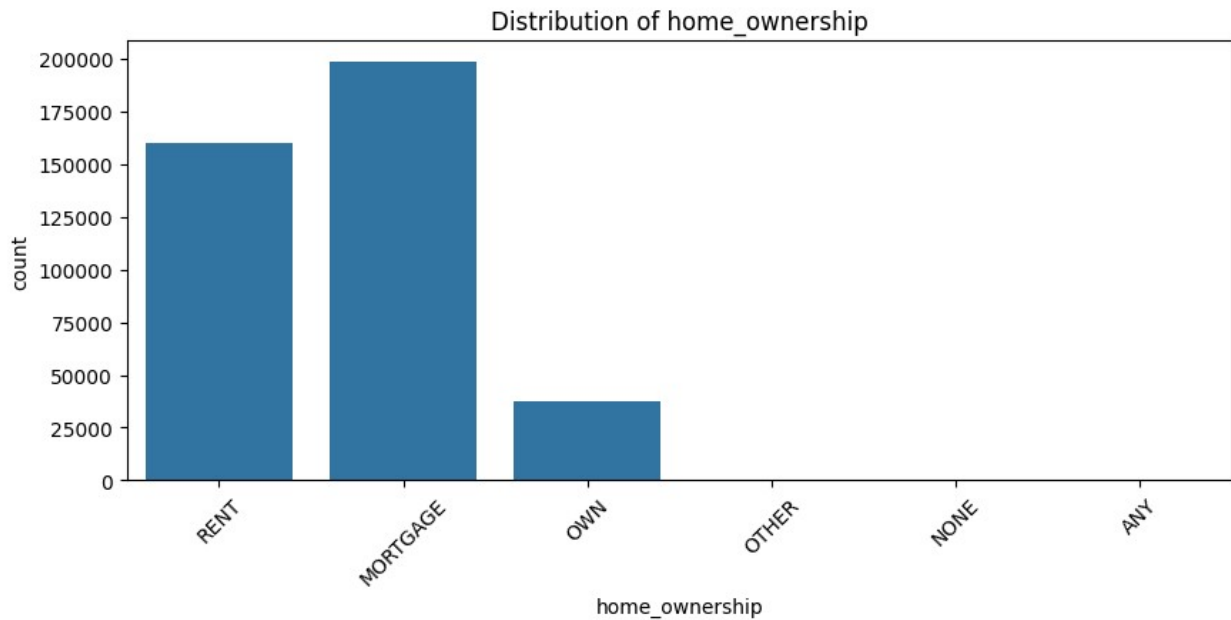
- The target variable (loan_status) is clearly imbalanced, with significantly more non-default (0) cases than default (1) cases.
- Most numerical features like annual_inc, dti, revol_util, open_acc, and total_acc are highly right-skewed with noticeable outliers.
- loan_amnt and installment show relatively structured distributions, but scaling and outlier treatment will be important before applying Logistic Regression.

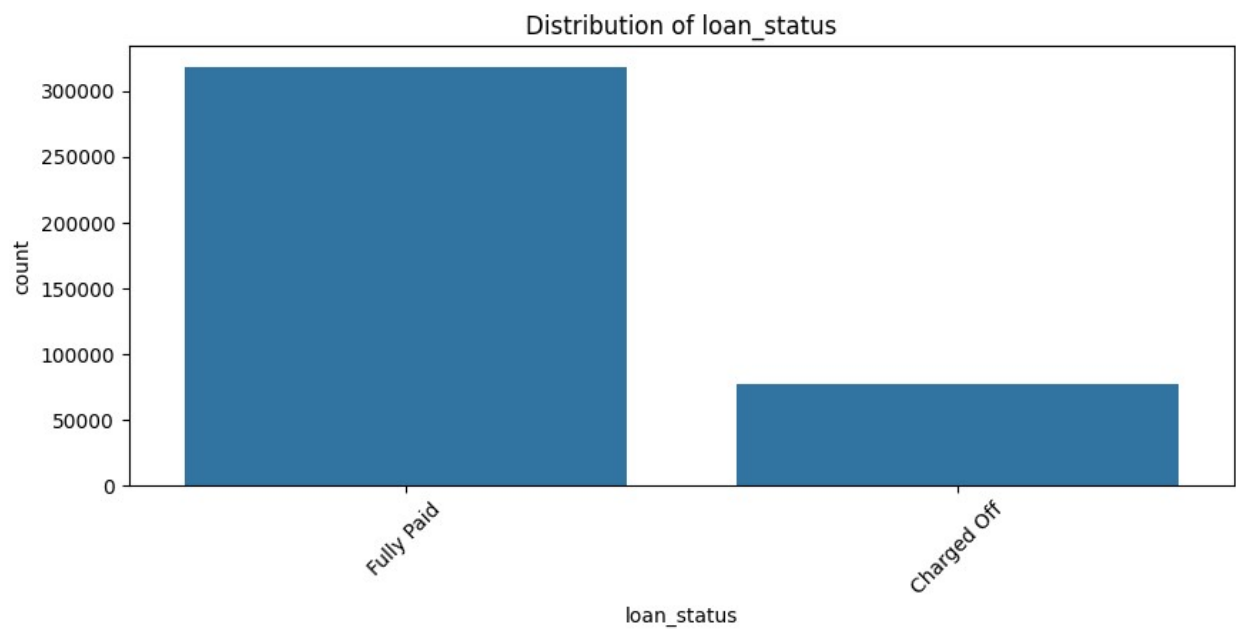
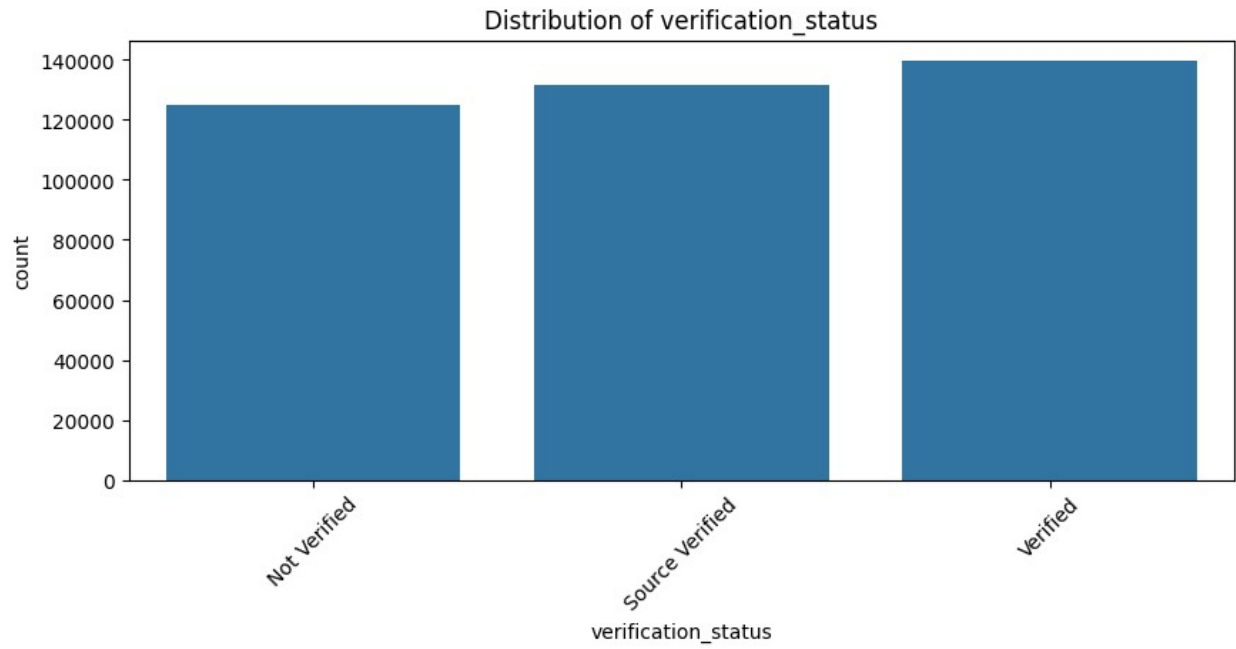
```
df['emp_title'].value_counts().head(10)
```

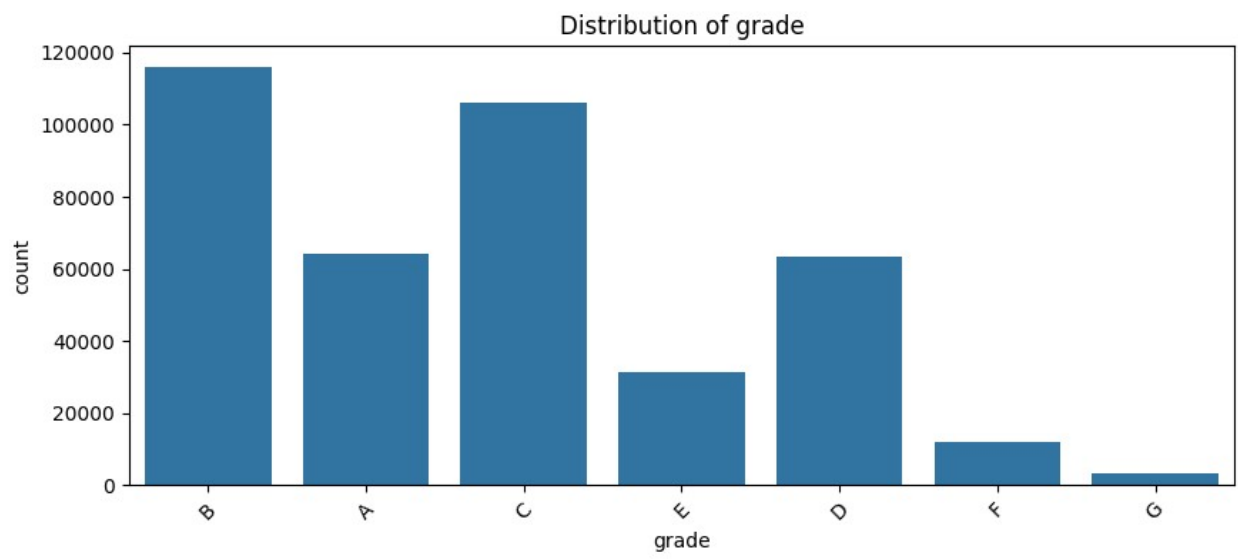
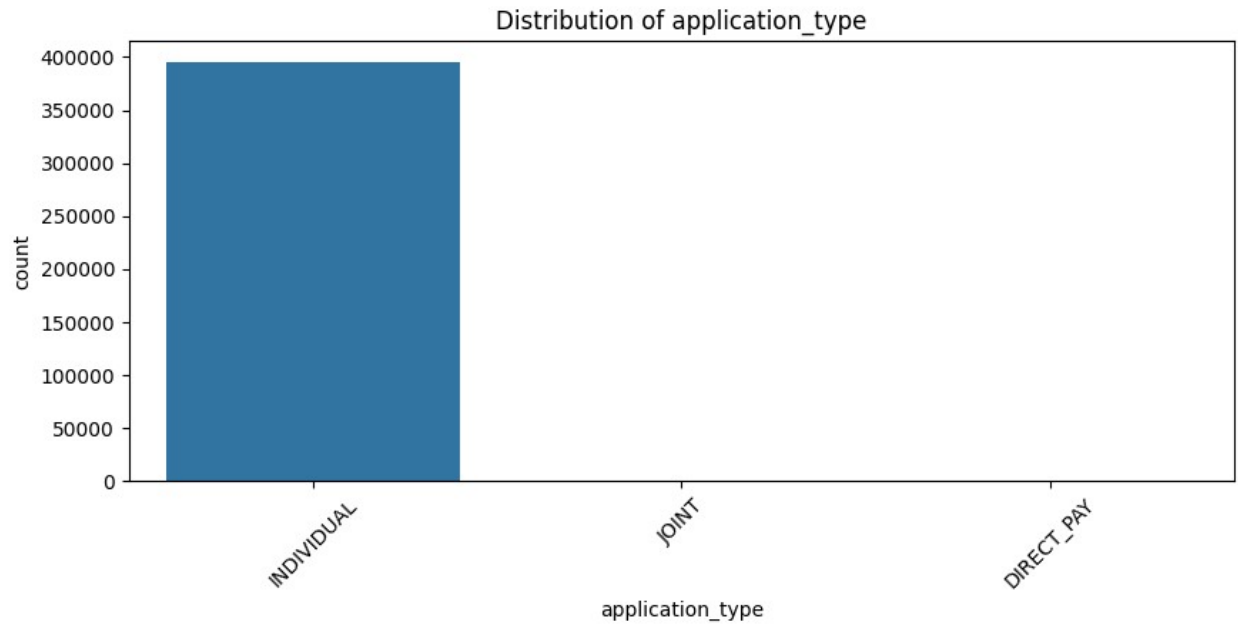
emp_title	
Teacher	4389
Manager	4250
Registered Nurse	1856

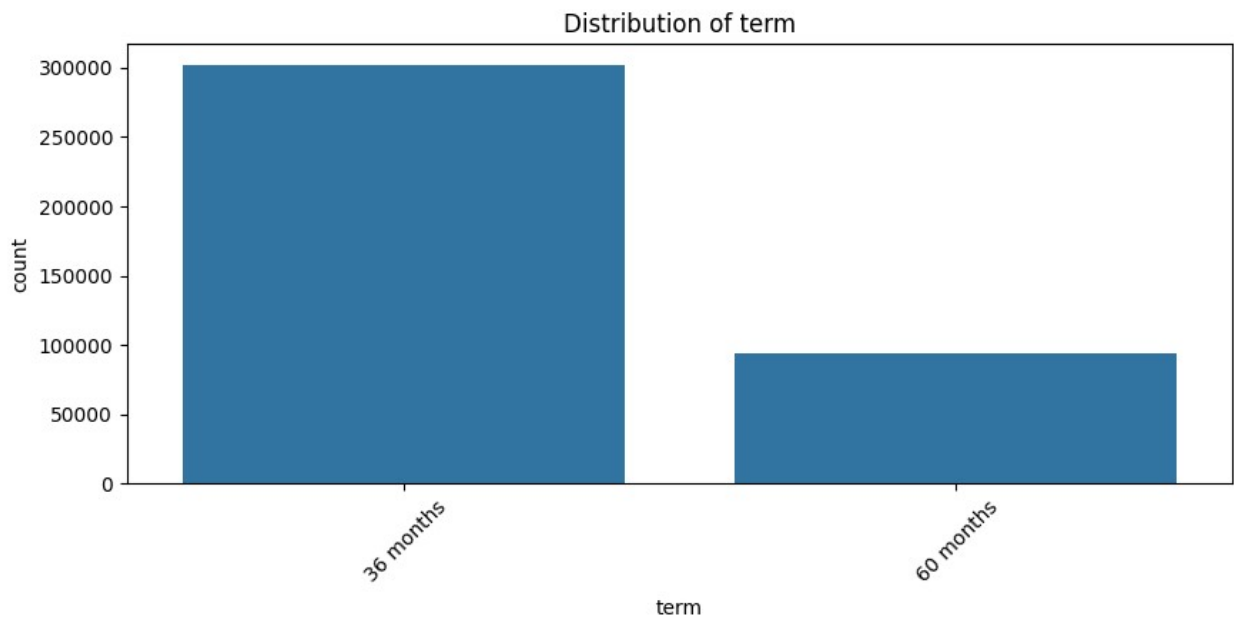
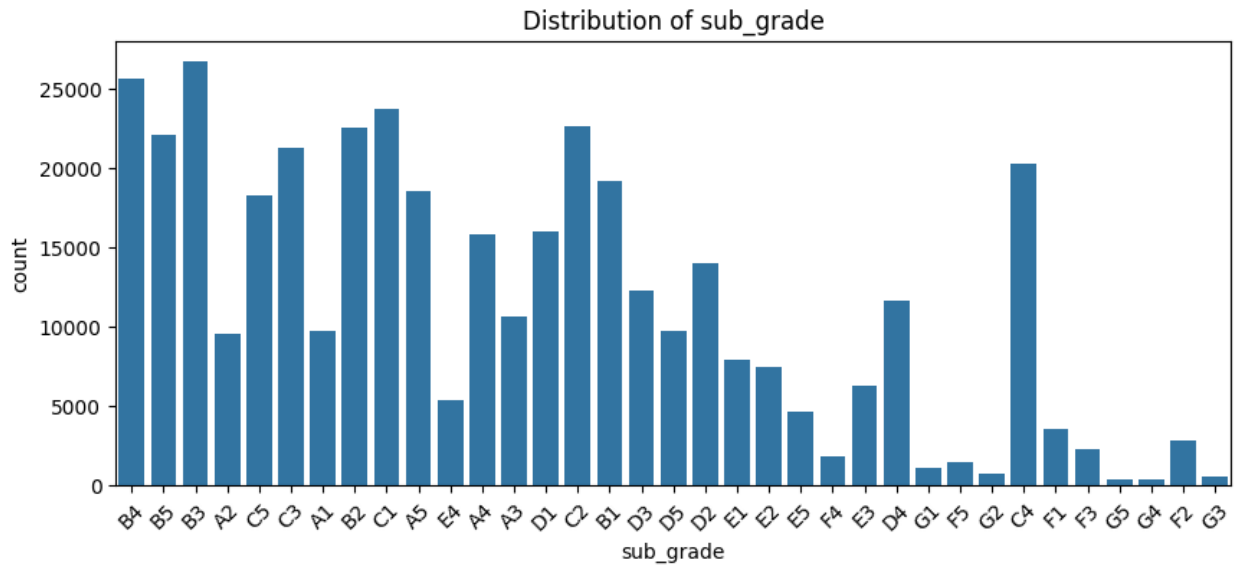
```
RN      1846
Supervisor 1830
Sales      1638
Project Manager 1505
Owner      1410
Driver     1339
Office Manager 1218
Name: count, dtype: int64
```

```
cat_cols = ['home_ownership', 'verification_status', 'loan_status',
            'application_type', 'grade', 'sub_grade', 'term']
for i in cat_cols:
    plt.figure(figsize=(10,4))
    plt.title(f"Distribution of {i}")
    sns.countplot(data = df, x = i)
    plt.xticks(rotation = 45)
    plt.show()
```









Observations

- Most borrowers are either RENT or MORTGAGE holders, with very few in OTHER/NONE categories; verification status is fairly balanced but slightly higher for Verified.
- Loan portfolio is dominated by Fully Paid (0) loans and 36-month terms, indicating class imbalance and preference for shorter tenure loans.
- Majority loans fall under Grade B and C, while lower grades (F, G) are minimal, showing concentration in mid-risk segments.

#Bivariate Analysis

```
plt.figure(figsize=(15,20))
```

```
plt.subplot(2,2,1)
```

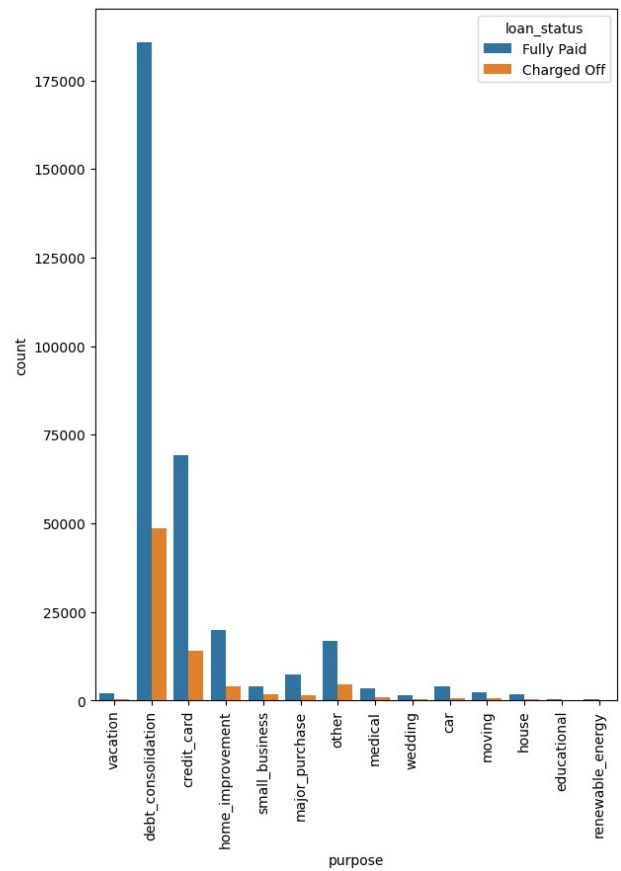
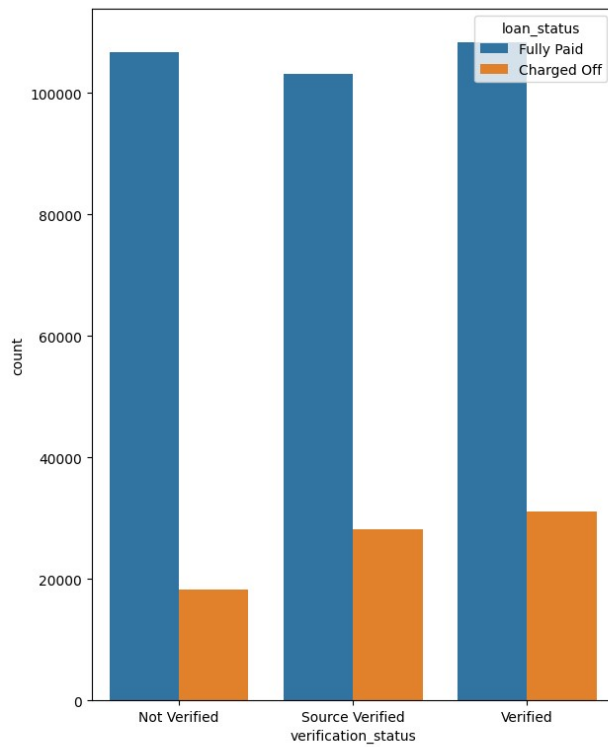
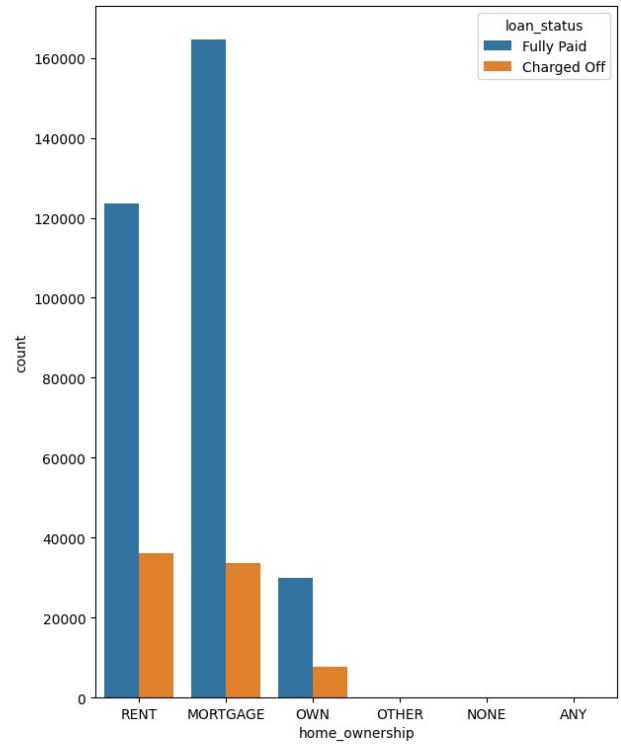
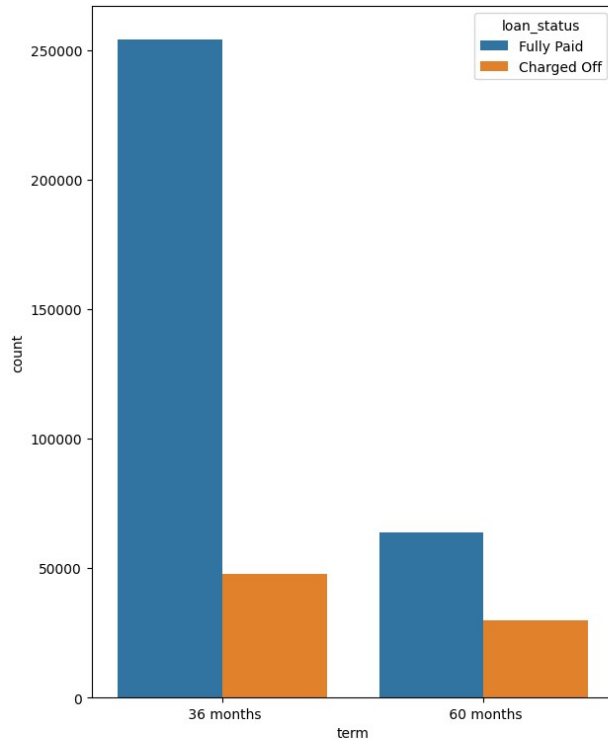
```
sns.countplot(x='term',data=df,hue='loan_status')

plt.subplot(2,2,2)
sns.countplot(x='home_ownership',data=df,hue='loan_status')

plt.subplot(2,2,3)
sns.countplot(x='verification_status',data=df,hue='loan_status')

plt.subplot(2,2,4)
g=sns.countplot(x='purpose',data=df,hue='loan_status')
g.set_xticklabels(g.get_xticklabels(),rotation=90)

plt.show()
```

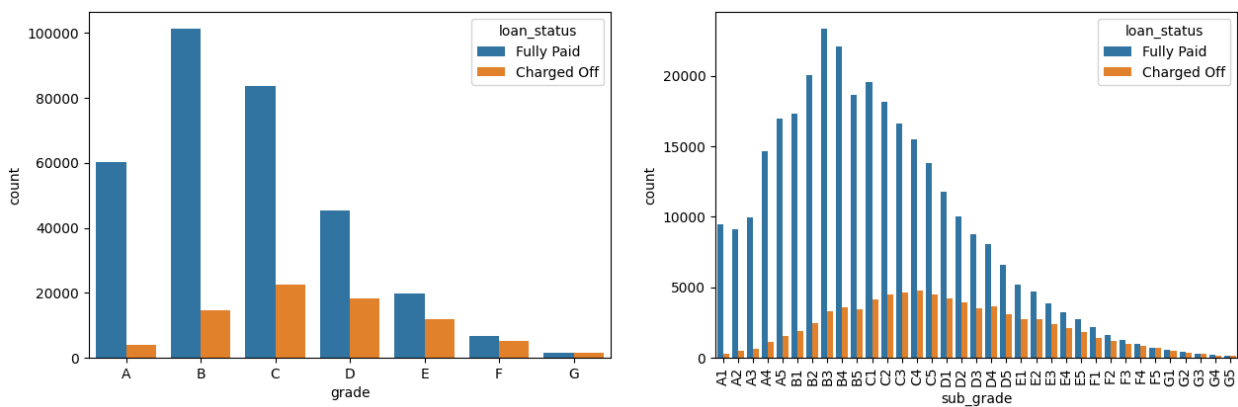


```
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
grade = sorted(df.grade.unique().tolist())
sns.countplot(x='grade', data=df, hue='loan_status', order=grade)

plt.subplot(2, 2, 2)
sub_grade = sorted(df.sub_grade.unique().tolist())
g = sns.countplot(x='sub_grade', data=df, hue='loan_status',
order=sub_grade)
g.set_xticklabels(g.get_xticklabels(), rotation=90)

plt.show()
```

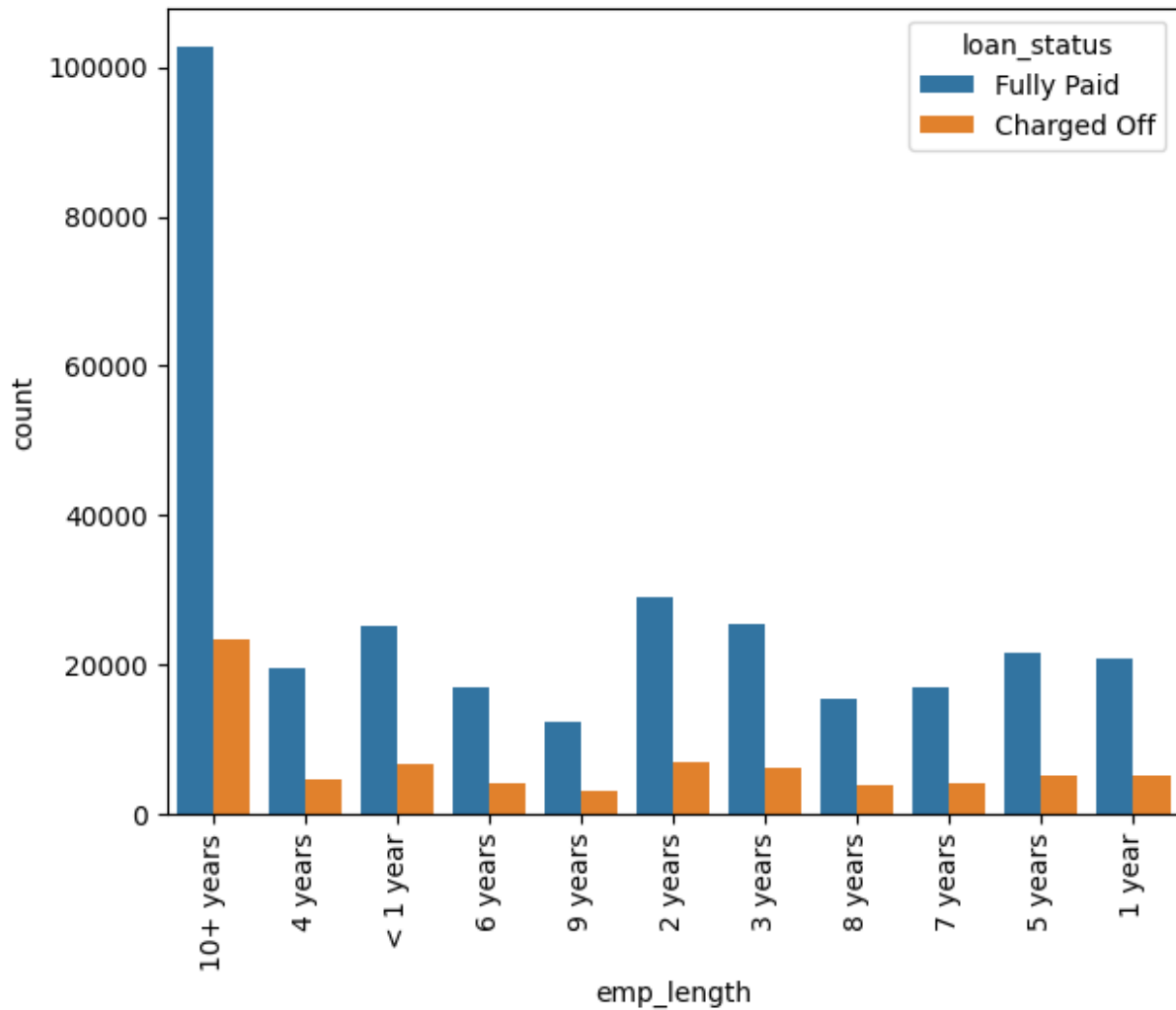


```
plt.figure(figsize=(15,12))

plt.subplot(2,2,1)

g=sns.countplot(x='emp_length',data=df,hue='loan_status')
g.set_xticklabels(g.get_xticklabels(),rotation=90)

plt.show()
```

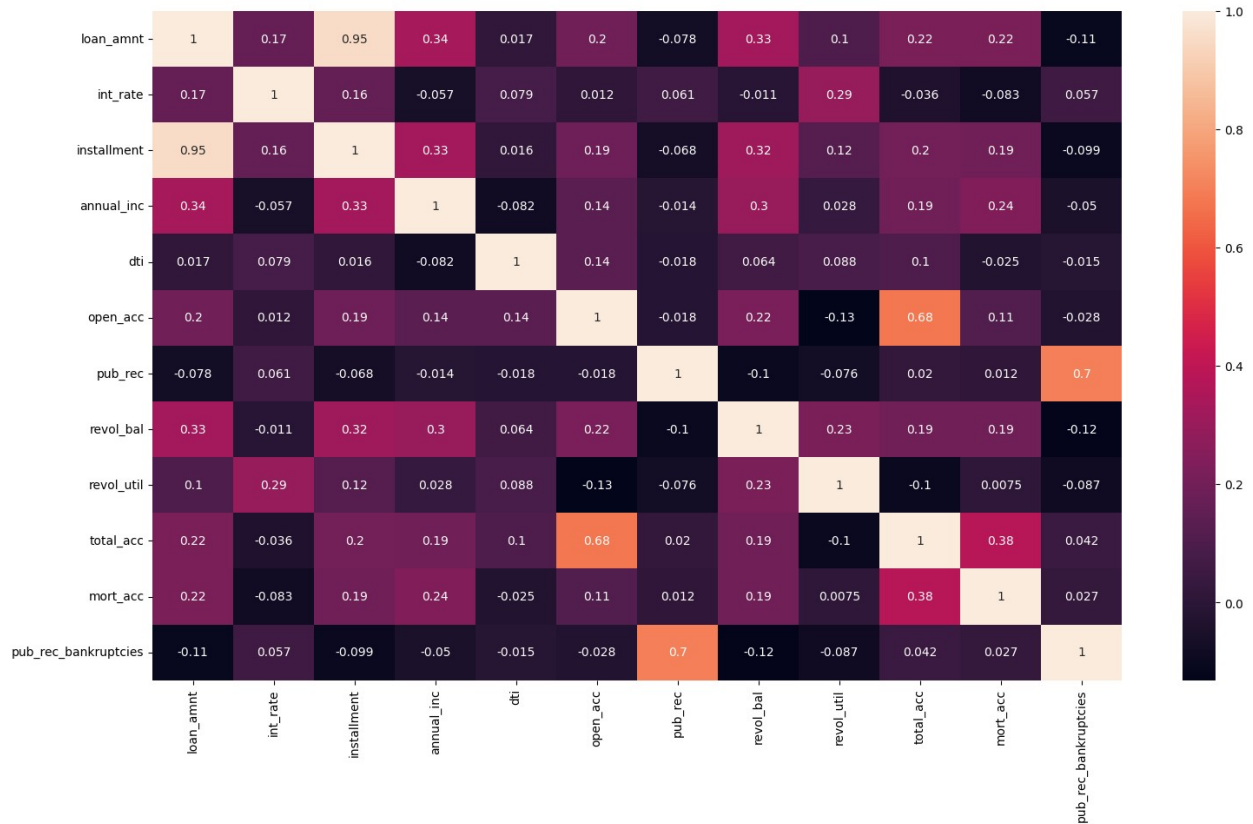


Observations

- **Loan Terms:** The most popular loan term is 36 months, with a high completion rate.
- **Loan Types:** Mortgages and rental loans are the most common loan types. Debt consolidation loans are also frequently used.
- **Creditworthiness:** Borrowers with a credit grade of "B" and a subgrade of "B3" tend to have the highest repayment rates.
- **Occupations:** Managers and teachers are the professions with the highest loan approval rates.
- **Repayment:** Individuals employed for over 10 years demonstrate a strong track record of loan repayment.

Correlation Analysis

```
plt.figure(figsize=(18,10))
sns.heatmap(df.corr(numeric_only=True), annot = True)
plt.show()
```



```
df.corr(numeric_only=True)
```

	loan_amnt	int_rate	installment	annual_inc
dti \				
loan_amnt	1.000000	0.168921	0.953929	0.336887
0.016636				
int_rate	0.168921	1.000000	0.162758	-0.056771
0.079038				
installment	0.953929	0.162758	1.000000	0.330381
0.015786				
annual_inc	0.336887	-0.056771	0.330381	1.000000
0.081685				
dti	0.016636	0.079038	0.015786	-0.081685
1.000000				
open_acc	0.198556	0.011649	0.188973	0.136150
0.136181				
pub_rec	-0.077779	0.060986	-0.067892	-0.013720
0.017639				
revol_bal	0.328320	-0.011280	0.316455	0.299773

0.063571				
revol_util	0.099911	0.293659	0.123915	0.027871
0.088375				
total_acc	0.223886	-0.036404	0.202430	0.193023
0.102128				
mort_acc	0.222315	-0.082583	0.193694	0.236320 -
0.025439				
pub_rec_bankruptcies	-0.106539	0.057450	-0.098628	-0.050162 -
0.014558				

	open_acc	pub_rec	revol_bal	revol_util	
total_acc \					
loan_amnt	0.198556	-0.077779	0.328320	0.099911	
0.223886					
int_rate	0.011649	0.060986	-0.011280	0.293659	-
0.036404					
installment	0.188973	-0.067892	0.316455	0.123915	
0.202430					
annual_inc	0.136150	-0.013720	0.299773	0.027871	
0.193023					
dti	0.136181	-0.017639	0.063571	0.088375	
0.102128					
open_acc	1.000000	-0.018392	0.221192	-0.131420	
0.680728					
pub_rec	-0.018392	1.000000	-0.101664	-0.075910	
0.019723					
revol_bal	0.221192	-0.101664	1.000000	0.226346	
0.191616					
revol_util	-0.131420	-0.075910	0.226346	1.000000	-
0.104273					
total_acc	0.680728	0.019723	0.191616	-0.104273	
1.000000					
mort_acc	0.109205	0.011552	0.194925	0.007514	
0.381072					
pub_rec_bankruptcies	-0.027732	0.699408	-0.124532	-0.086751	
0.042035					

	mort_acc	pub_rec_bankruptcies
loan_amnt	0.222315	-0.106539
int_rate	-0.082583	0.057450
installment	0.193694	-0.098628
annual_inc	0.236320	-0.050162
dti	-0.025439	-0.014558
open_acc	0.109205	-0.027732
pub_rec	0.011552	0.699408
revol_bal	0.194925	-0.124532
revol_util	0.007514	-0.086751
total_acc	0.381072	0.042035

mort_acc	1.000000	0.027239
pub_rec_bankruptcies	0.027239	1.000000

▯ Observations

- Positive correlation with annual income (annual_inc) - Higher income allows for larger loan applications.
- Weak positive correlation with installment amount (installment) - Makes sense as larger loans will typically have higher installments.
- Weak positive correlation with total accounts (total_acc) and mortgage accounts (mort_acc) - Borrowers with more established credit lines may be eligible for higher loan amounts.
- Weak negative correlation with annual income (annual_inc) - Generally, borrowers with higher income qualify for lower interest rates.
- Weak positive correlation with total accounts (total_acc) and mortgage accounts (mort_acc) - People with a higher income may tend to have more credit accounts.
- Positive correlation between revolving balance (revol_bal) and credit line utilization (revol_util) - This indicates that people with higher credit balances also tend to have a higher utilization ratio.
- Weak positive correlation between number of open accounts (open_acc) and total accounts (total_acc) - As expected, people with more open accounts tend to have more total accounts.

```
def pub_rec(number):
    if number == 0.0:
        return 0
    else:
        return 1

def mort_acc(number):
    if number == 0.0:
        return 0
    elif number >= 1.0:
        return 1
    else:
        return number

def pub_rec_bankruptcies(number):
    if number == 0.0:
        return 0
    elif number >= 1.0:
        return 1
```

```
    else:
        return number

df['pub_rec']=df.pub_rec.apply(pub_rec)
df['mort_acc']=df.mort_acc.apply(mort_acc)
df['pub_rec_bankruptcies']=df.pub_rec_bankruptcies.apply(pub_rec_bankruptcies)

plt.figure(figsize=(12,30))

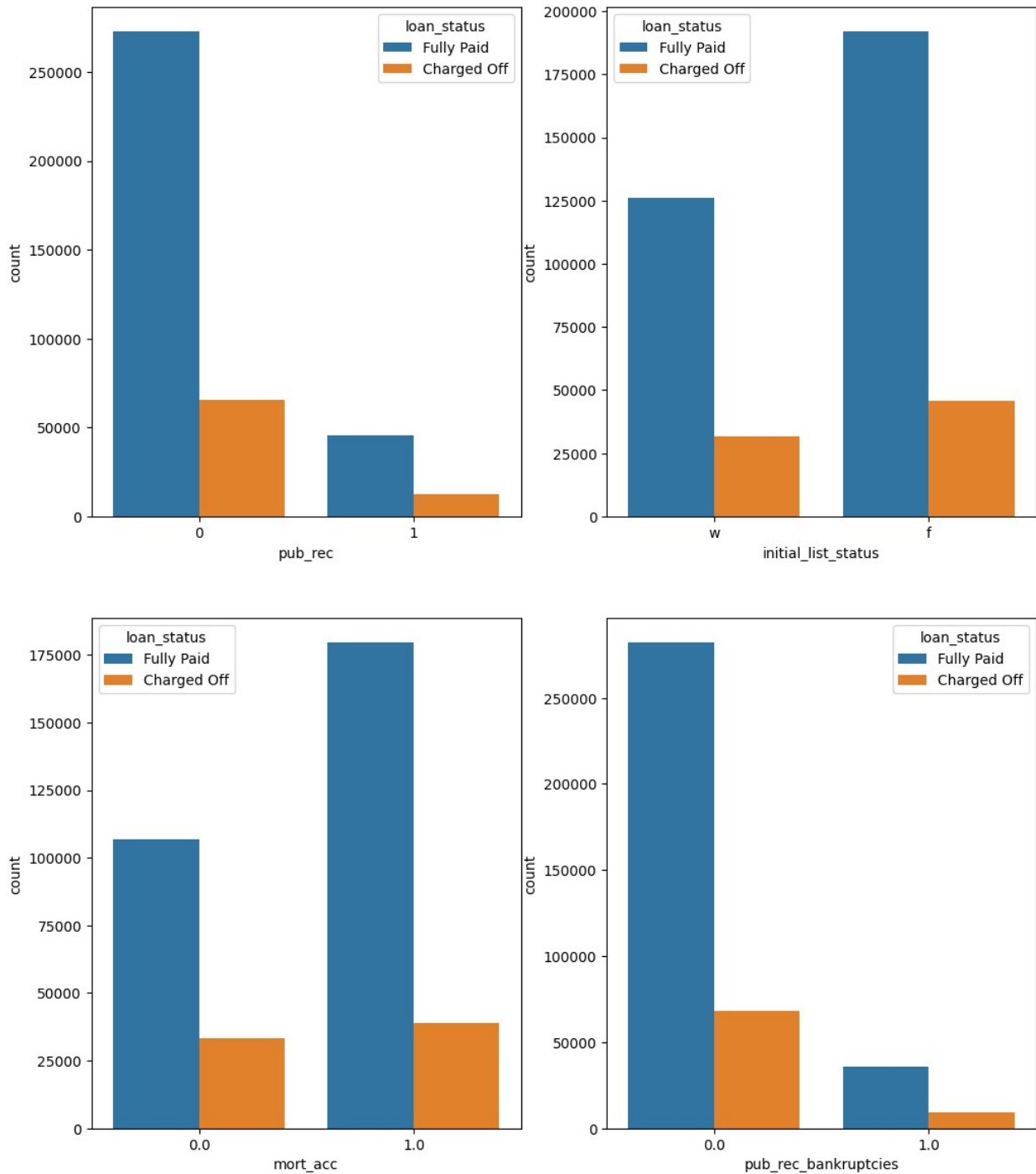
plt.subplot(4,2,1)
sns.countplot(x='pub_rec',data=df,hue='loan_status')

plt.subplot(4,2,2)
sns.countplot(x='initial_list_status',data=df,hue='loan_status')

plt.subplot(4,2,3)
sns.countplot(x='mort_acc',data=df,hue='loan_status')

plt.subplot(4,2,4)
sns.countplot(x='pub_rec_bankruptcies',data=df,hue='loan_status')

plt.show()
```



Most the loan disbursed to the people who do not hold bankruptcies record have successfully paid loan

Duplicate and null values

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df.isnull().sum()
```

```
loan_amnt      0
term           0
int_rate       0
installment    0
grade          0
sub_grade      0
emp_title      22927
emp_length     18301
home_ownership  0
annual_inc     0
verification_status  0
issue_d        0
loan_status    0
purpose        0
title          1756
dti            0
earliest_cr_line  0
open_acc       0
pub_rec        0
revol_bal      0
revol_util     276
total_acc      0
initial_list_status  0
application_type  0
mort_acc       37795
pub_rec_bankruptcies  535
address        0
dtype: int64
```

```
numeric_columns = df.select_dtypes(include=['float64', 'int64'])
```

```
total_acc_avg = numeric_columns.groupby('total_acc')
['mort_acc'].mean()
```

```
def fill_mort_acc(total_acc, mort_acc):
    if np.isnan(mort_acc):
        return total_acc_avg[total_acc].round()
    else:
        return mort_acc
```

```
df['mort_acc'] = df.apply(lambda x: fill_mort_acc(x['total_acc'],
x['mort_acc']), axis=1)
```

```
df.isnull().sum()
```

```
loan_amnt      0
term           0
int_rate       0
```

```

installment      0
grade            0
sub_grade        0
emp_title        22927
emp_length       18301
home_ownership   0
annual_inc       0
verification_status  0
issue_d          0
loan_status      0
purpose          0
title            1756
dti              0
earliest_cr_line  0
open_acc         0
pub_rec          0
revol_bal        0
revol_util       276
total_acc        0
initial_list_status  0
application_type  0
mort_acc         0
pub_rec_bankruptcies  535
address          0
dtype: int64

# dropping remaining null values

df.dropna(inplace=True)

df.shape

(370621, 27)

```

Outlier Treatment

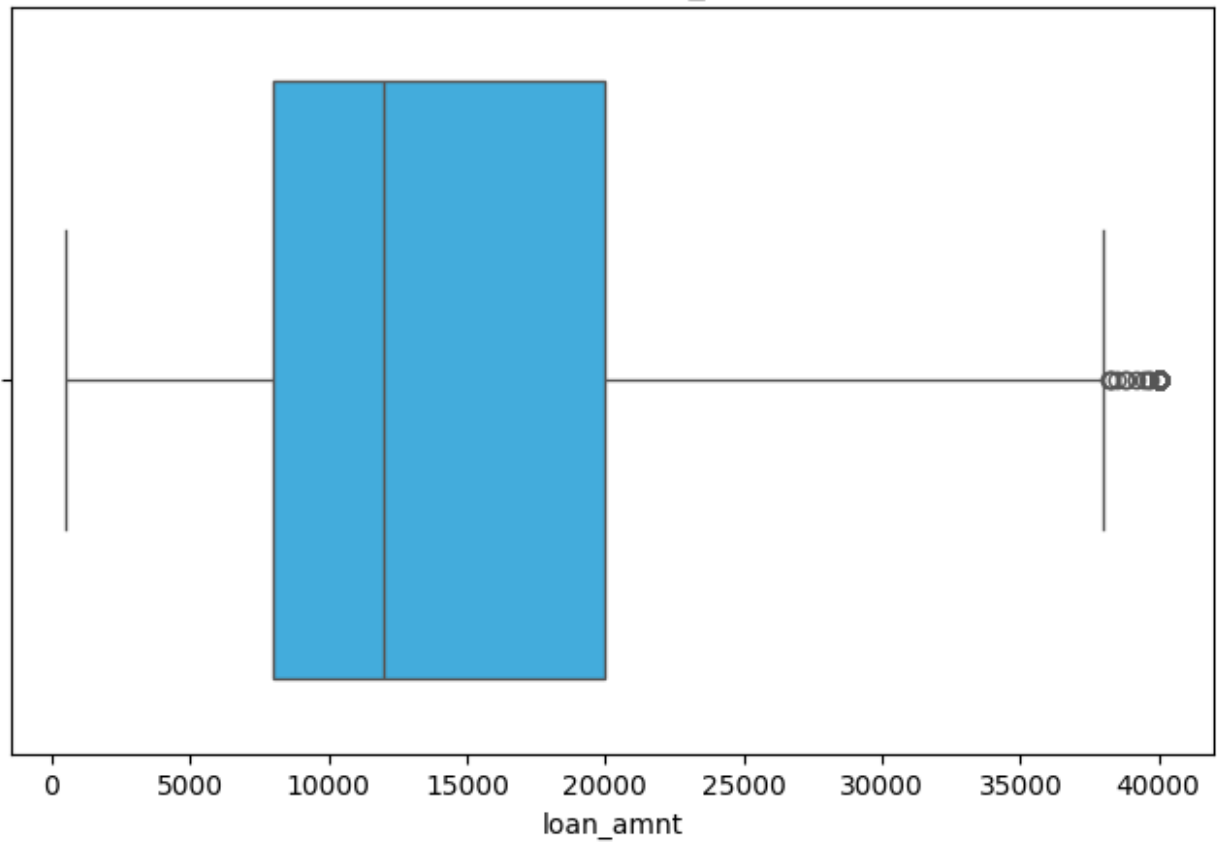
```

def box_plot(col):
    if col in df.columns:
        plt.figure(figsize=(8, 5))
        sns.boxplot(x=df[col], color="#29B6F6")
        plt.title('Boxplot for {}'.format(col))
        plt.show()
    else:
        print(f"Column '{col}' not found in the DataFrame.")

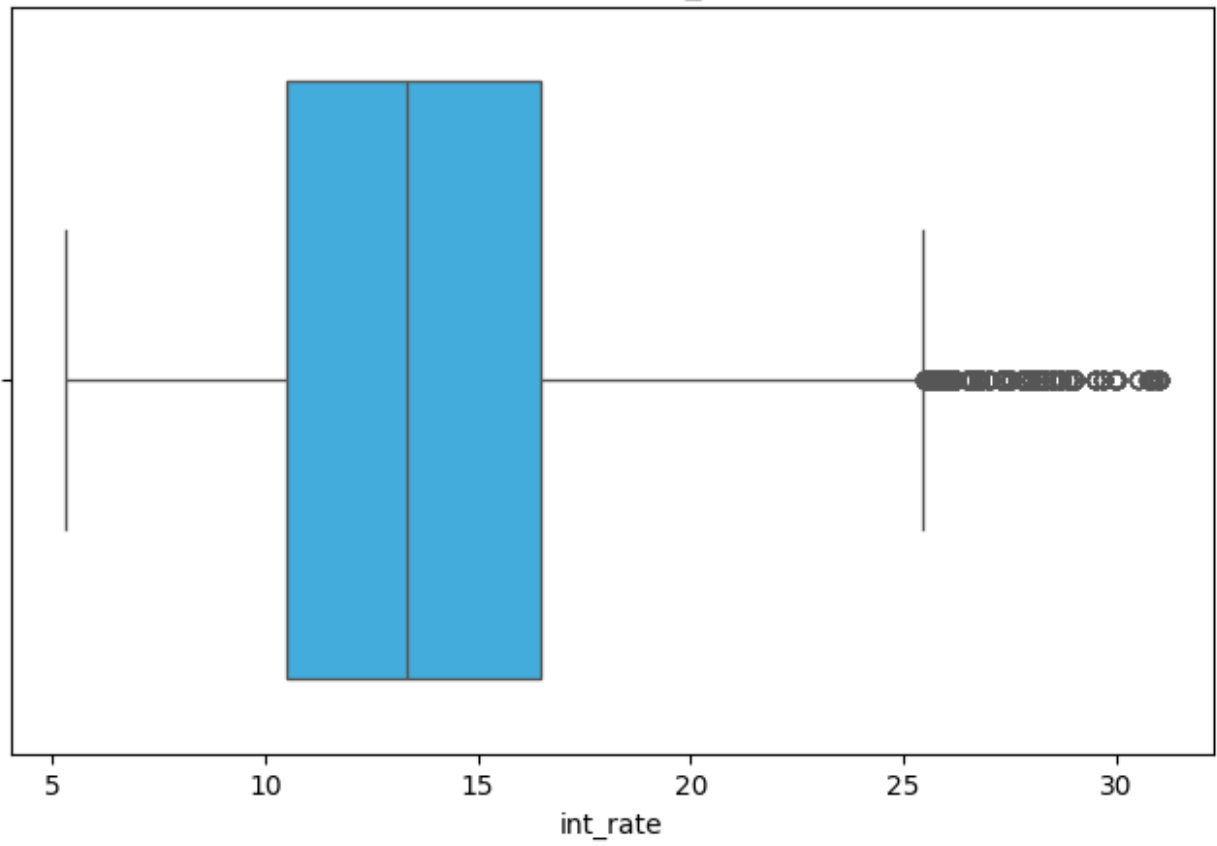
for col in num_cols:
    box_plot(col)

```

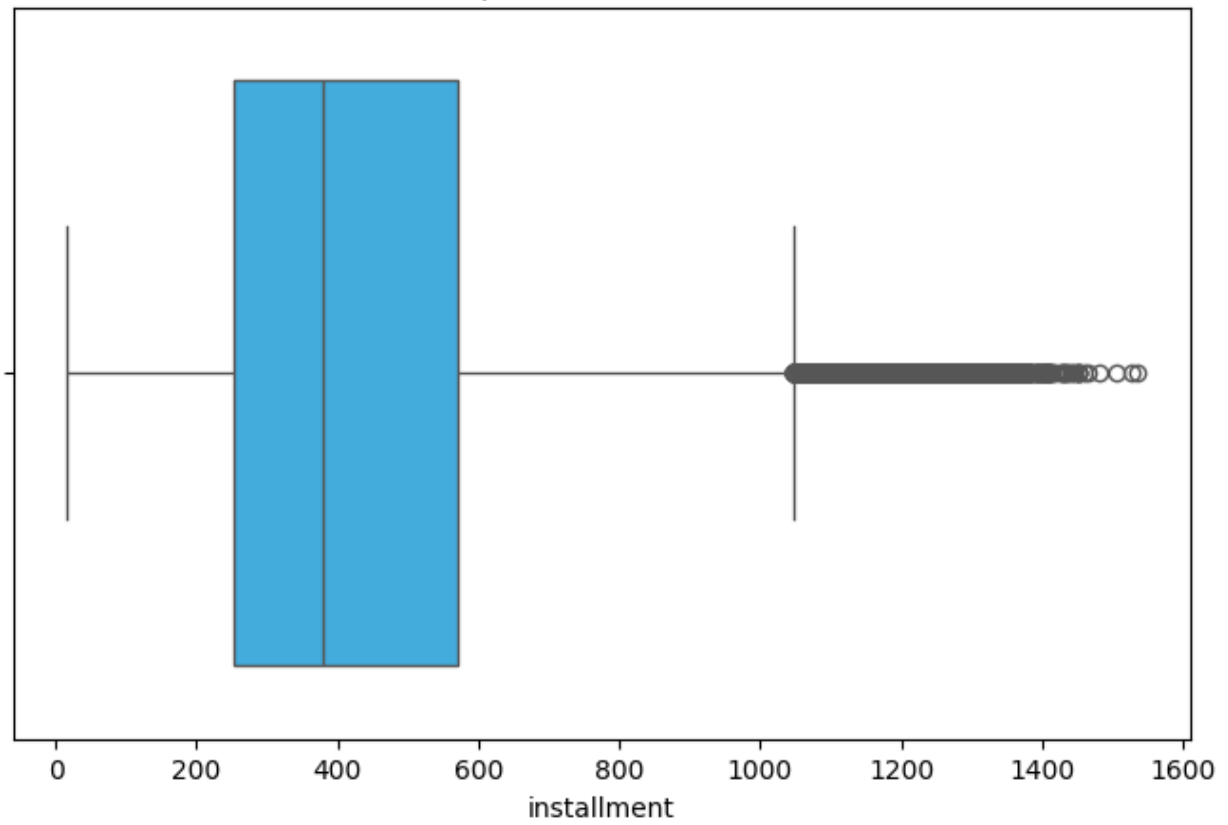
Boxplot for loan_amnt



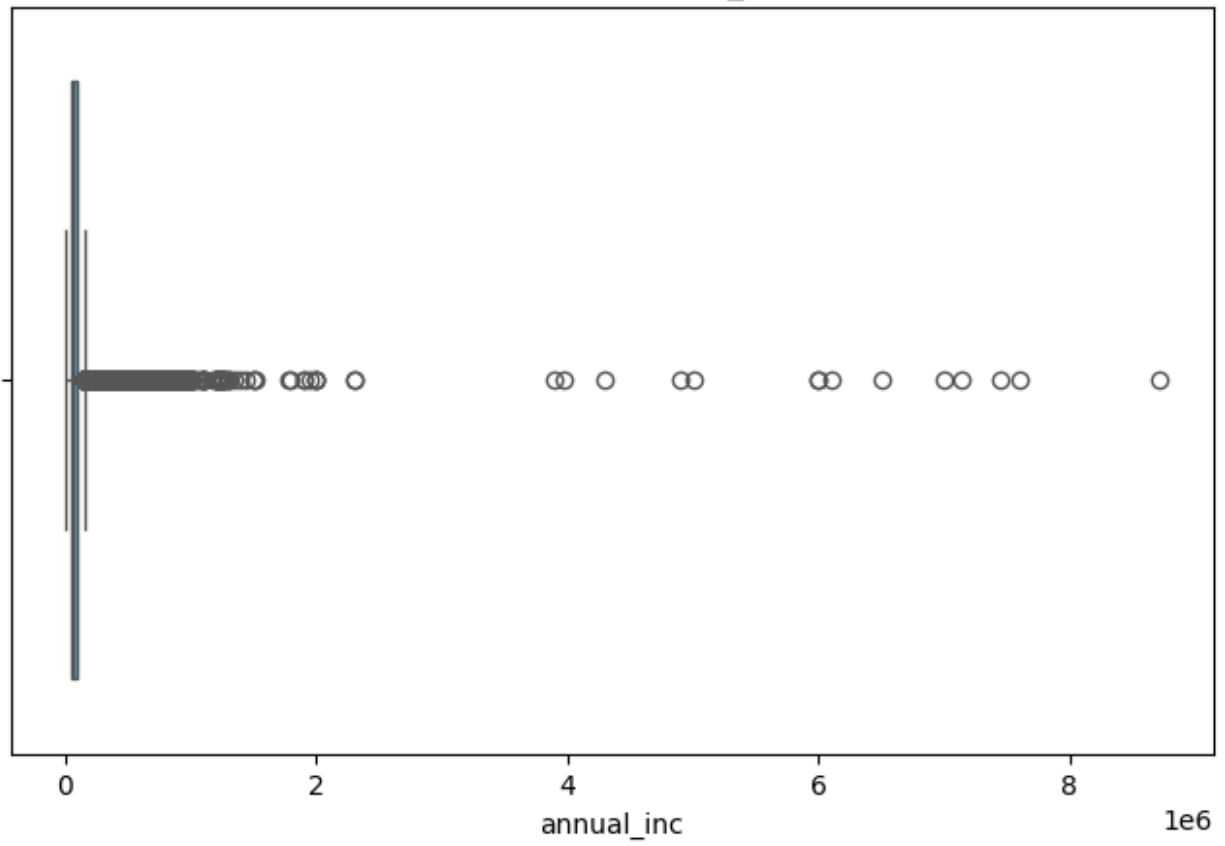
Boxplot for int_rate



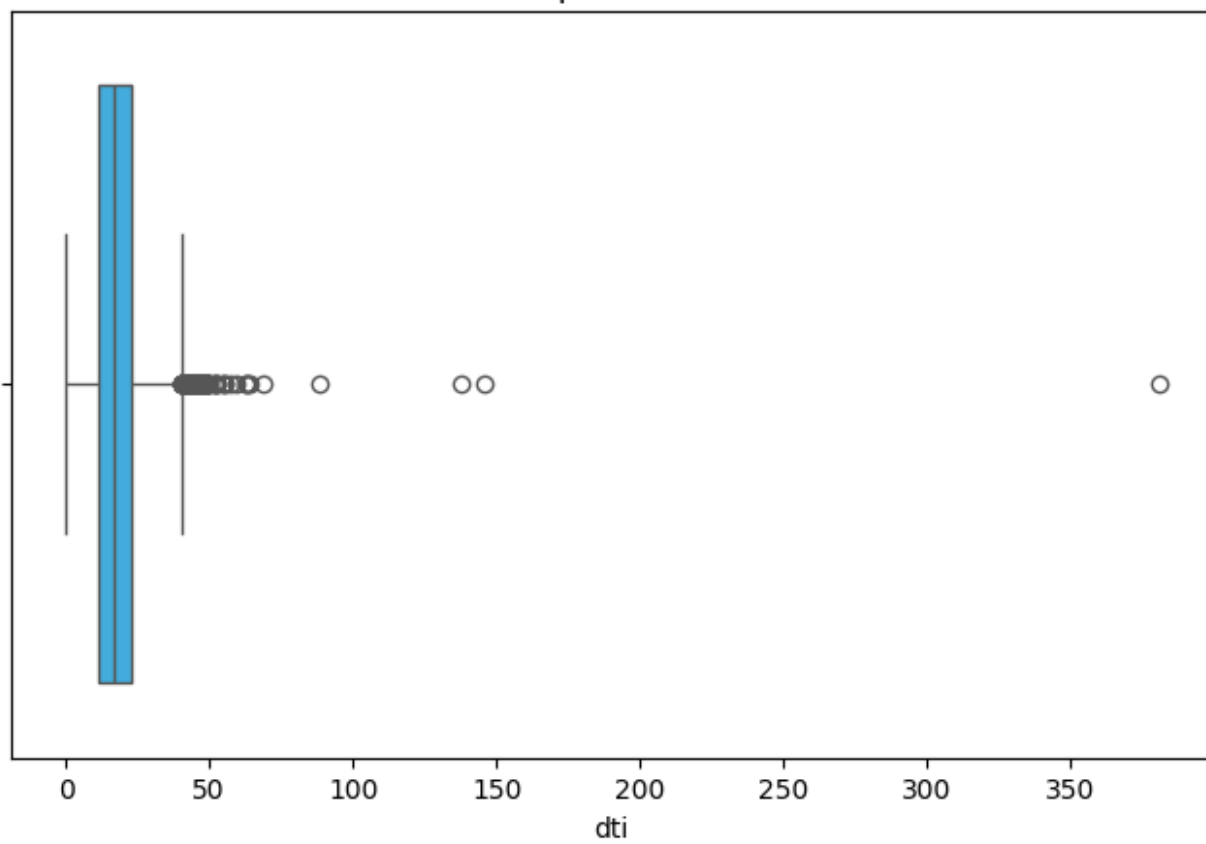
Boxplot for installment



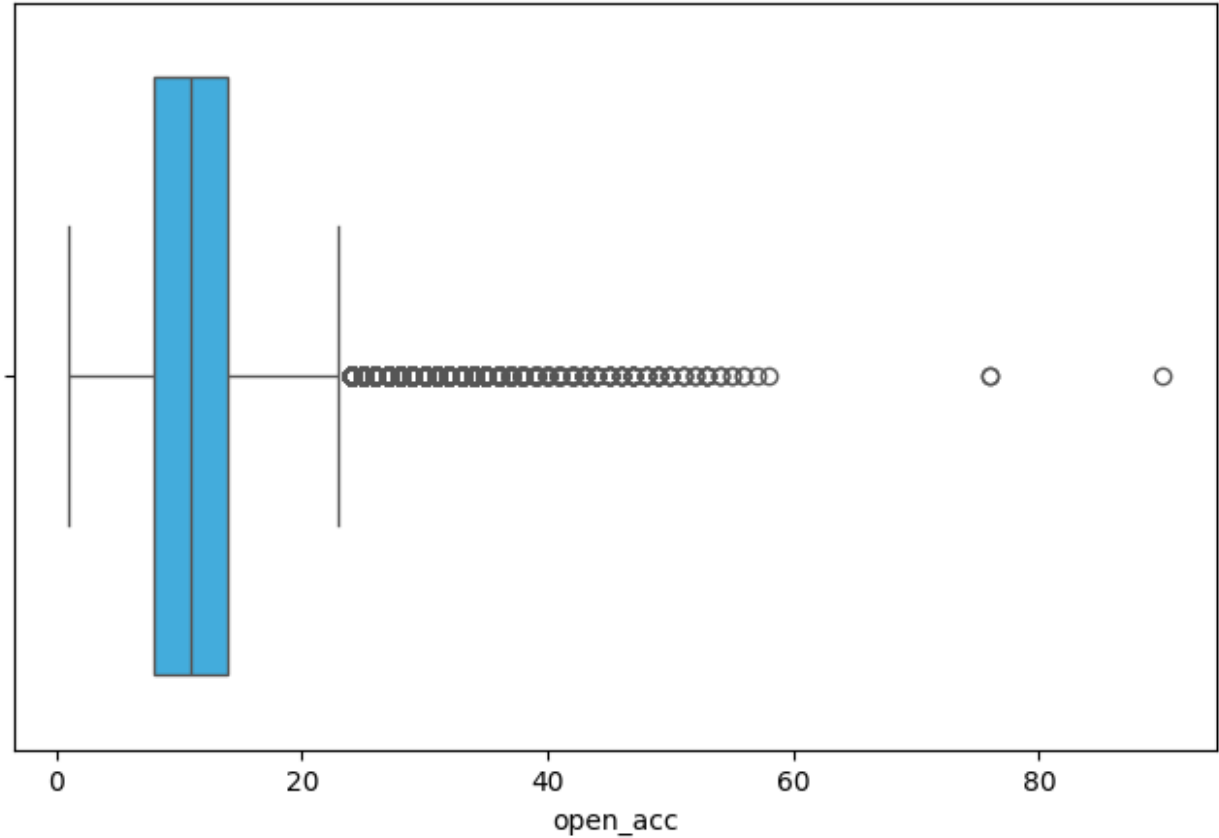
Boxplot for annual_inc



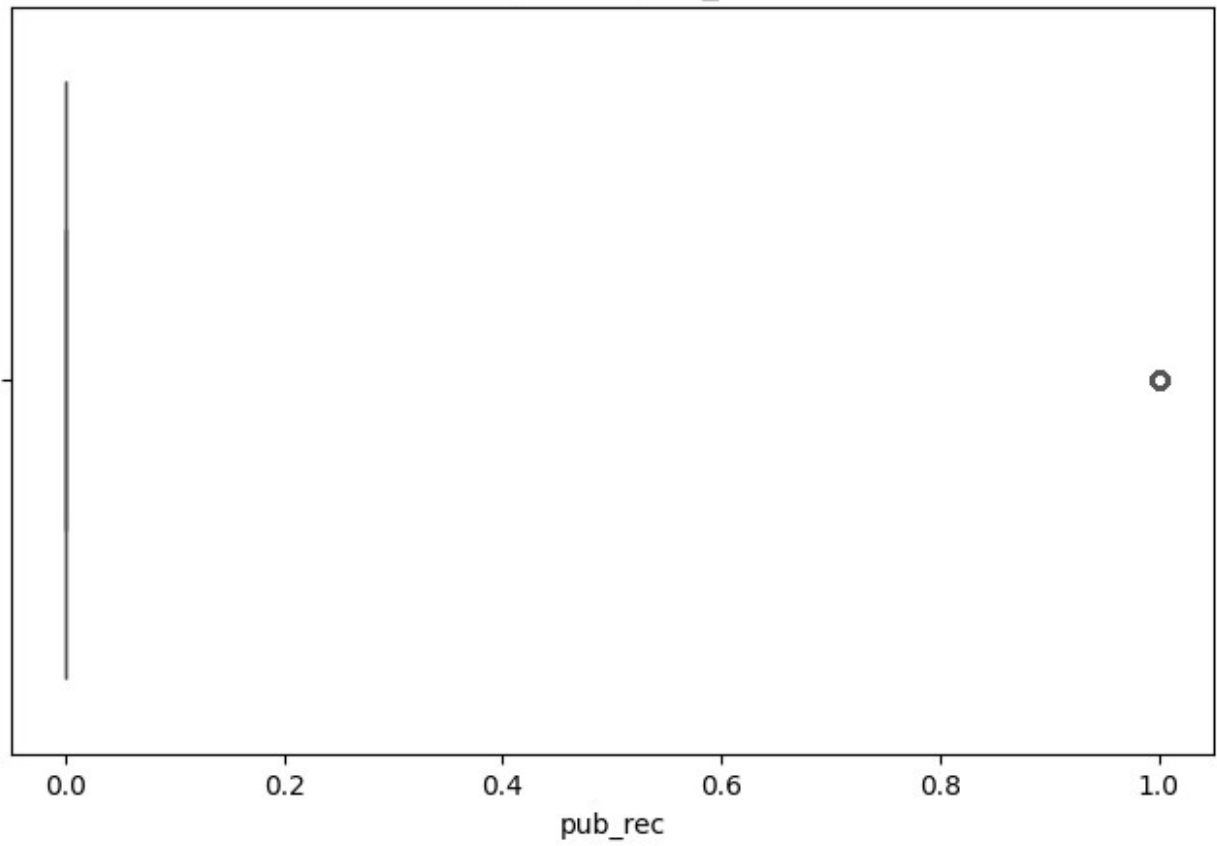
Boxplot for dti



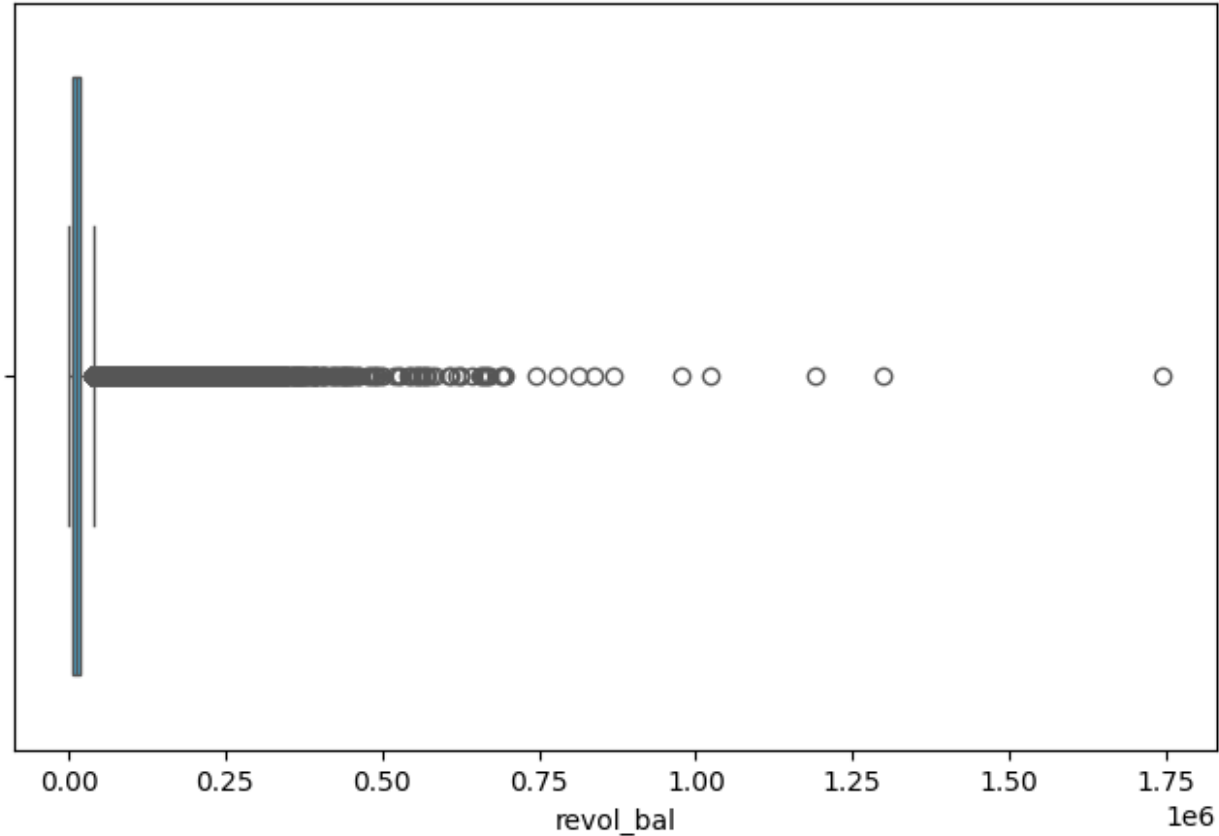
Boxplot for open_acc



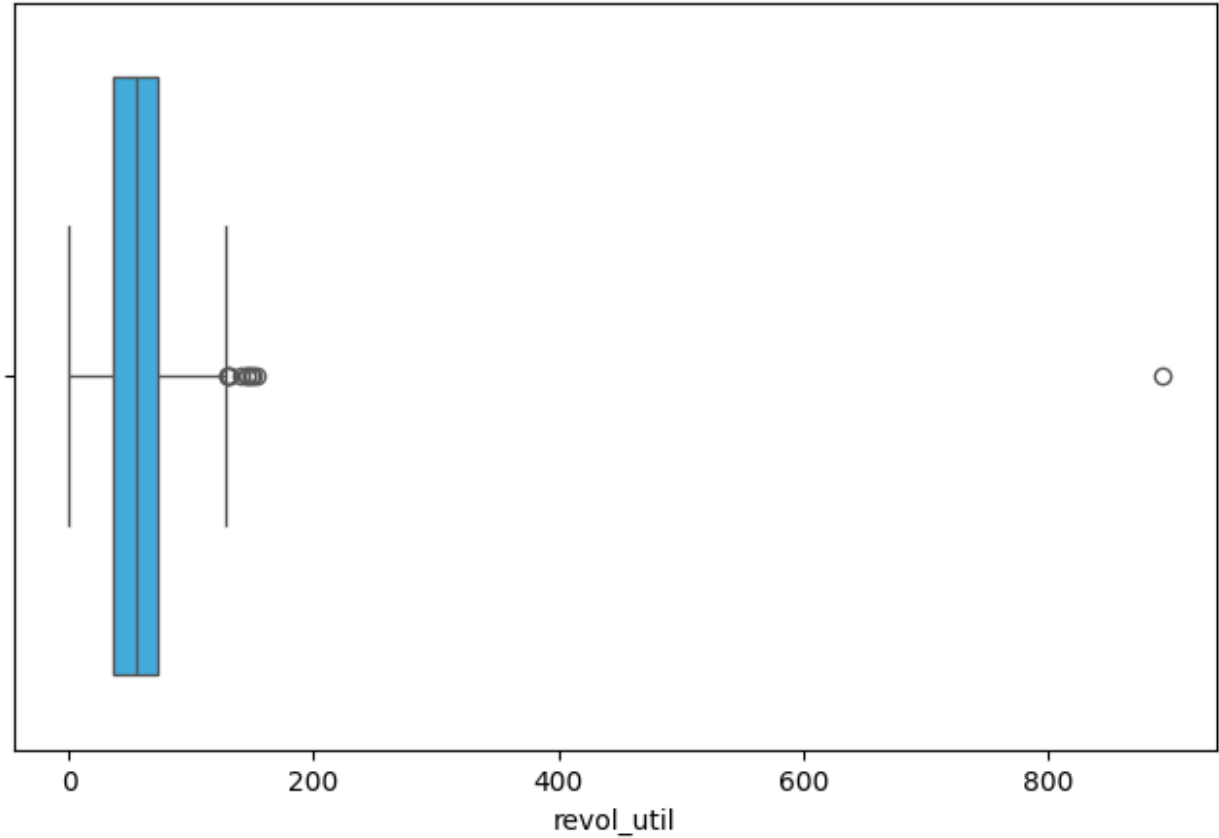
Boxplot for pub_rec



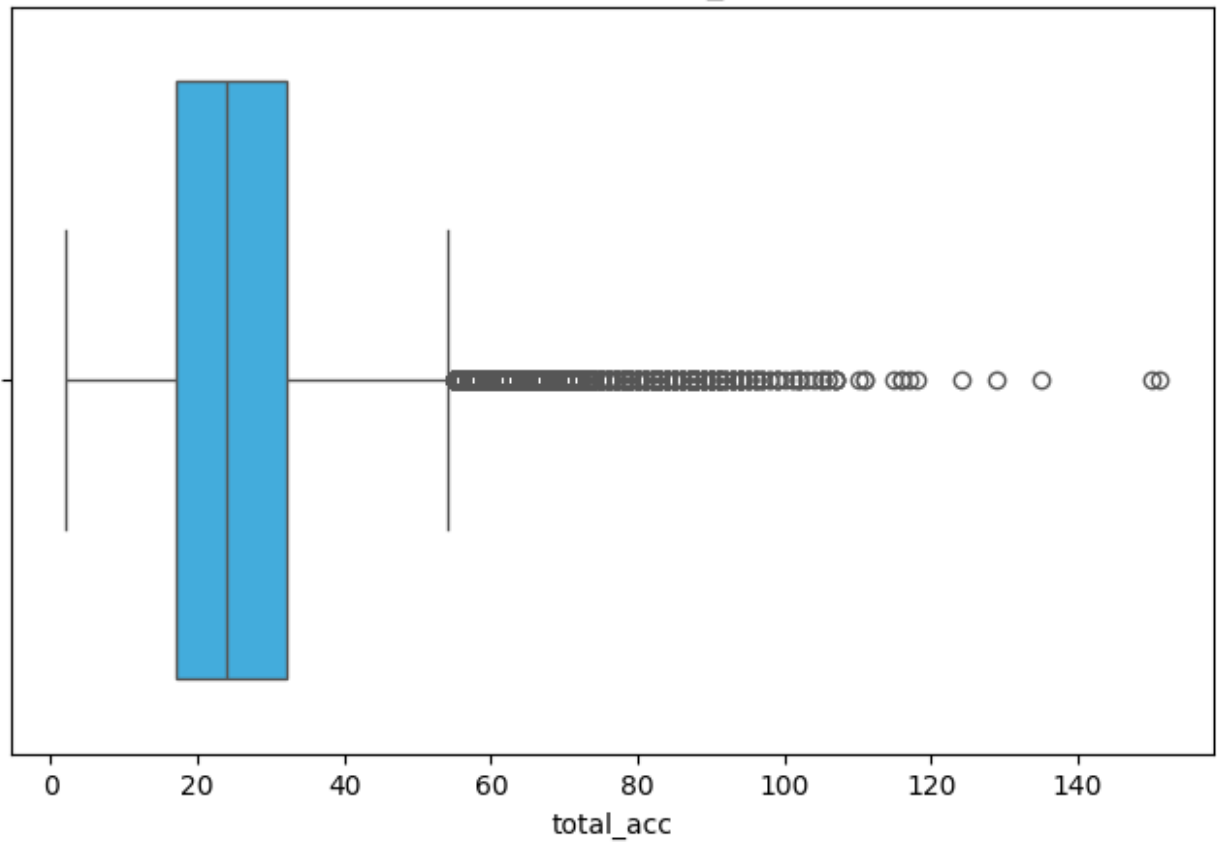
Boxplot for revol_bal



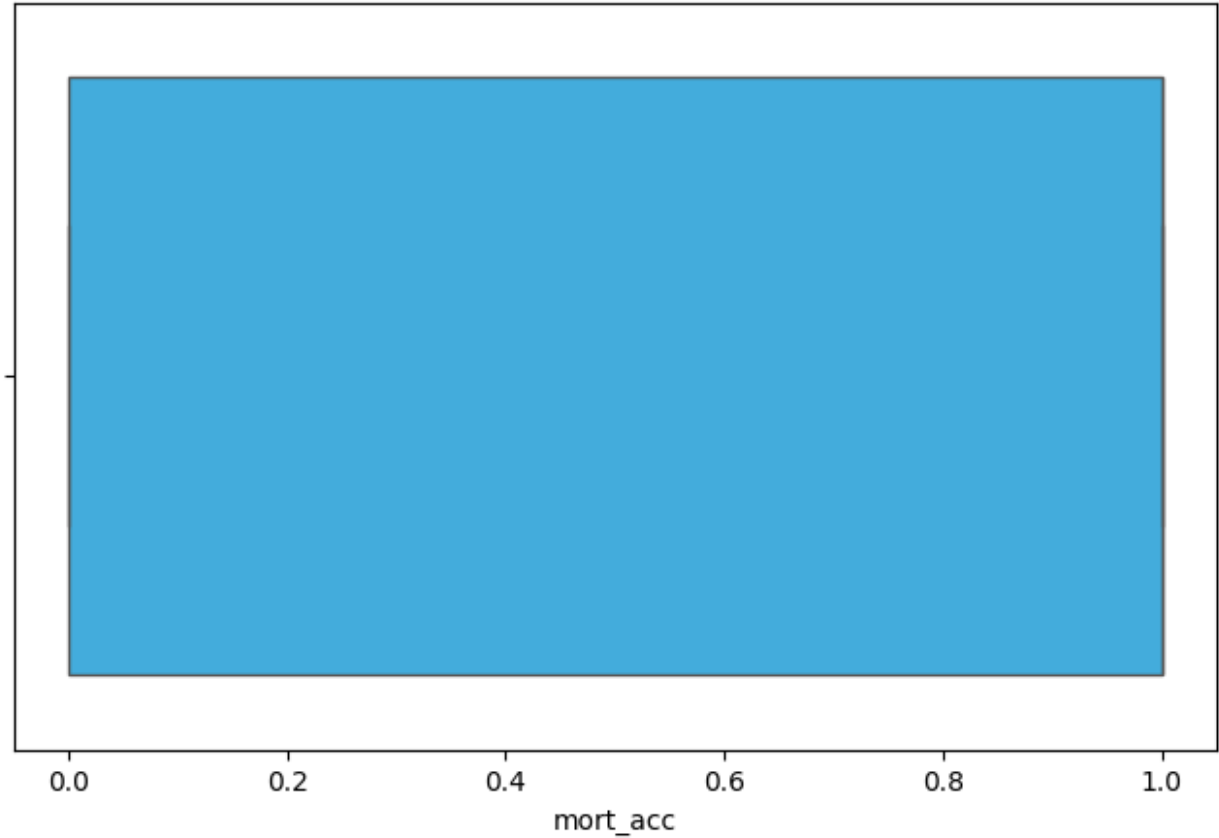
Boxplot for revol_util

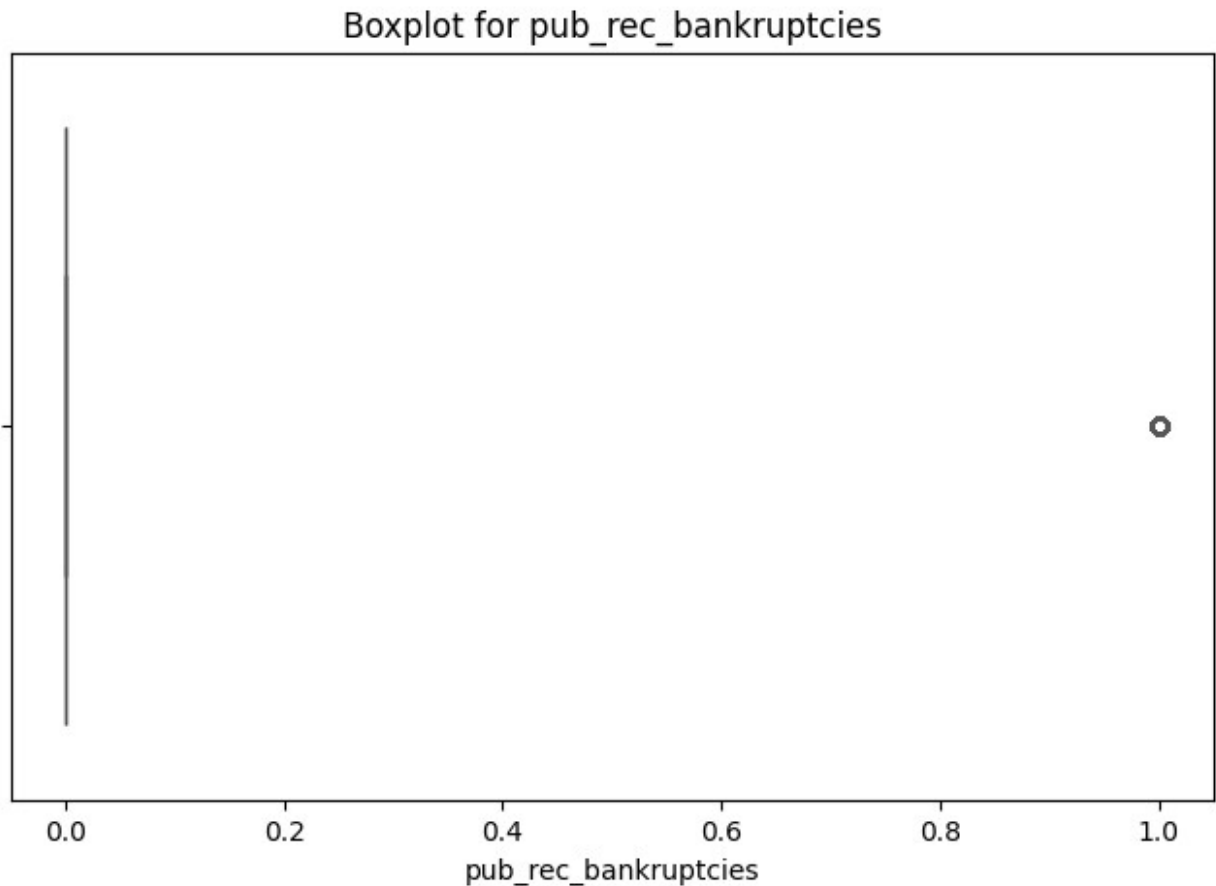


Boxplot for total_acc



Boxplot for mort_acc





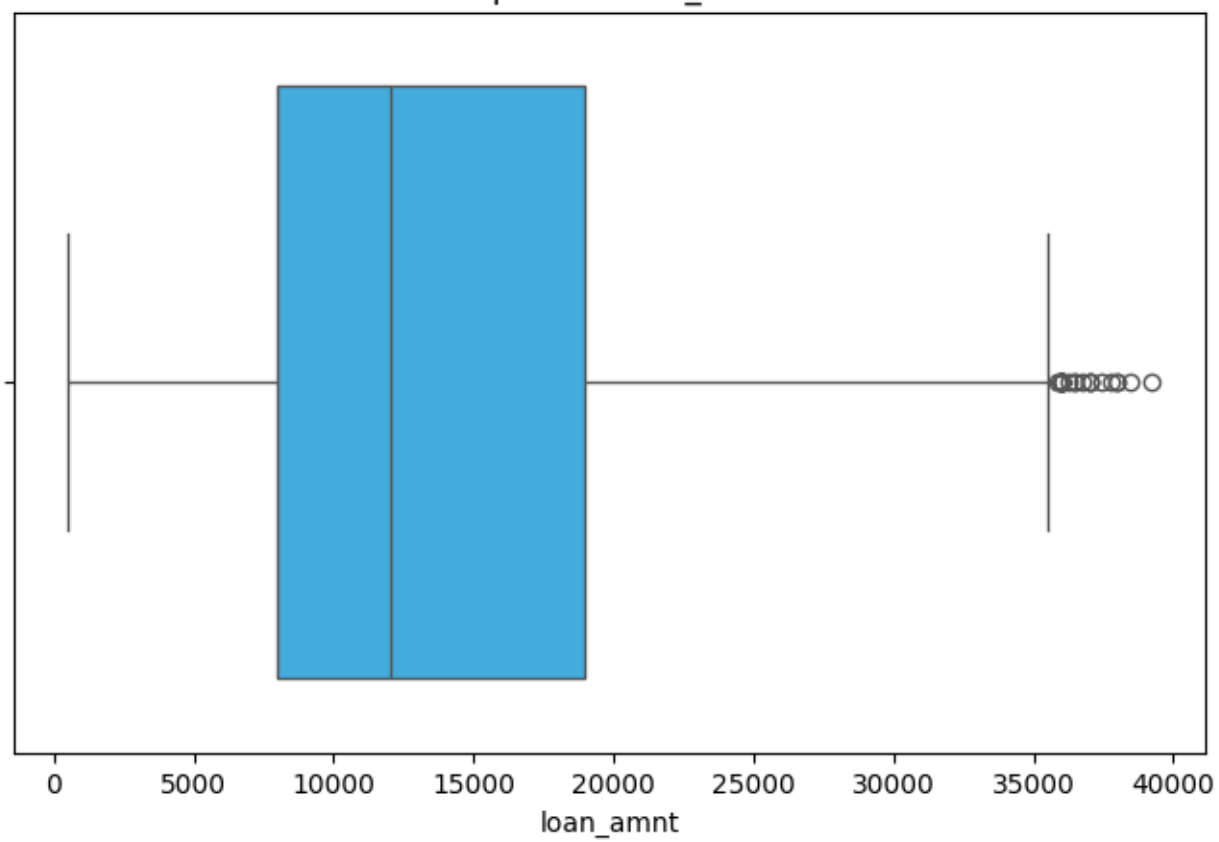
```
# Outlier treatment

for col in num_cols:
    if col in df.columns:
        mean = df[col].mean()
        std = df[col].std()
        upper_limit = mean + 3 * std
        lower_limit = mean - 3 * std
        df = df[(df[col] < upper_limit) & (df[col] > lower_limit)]

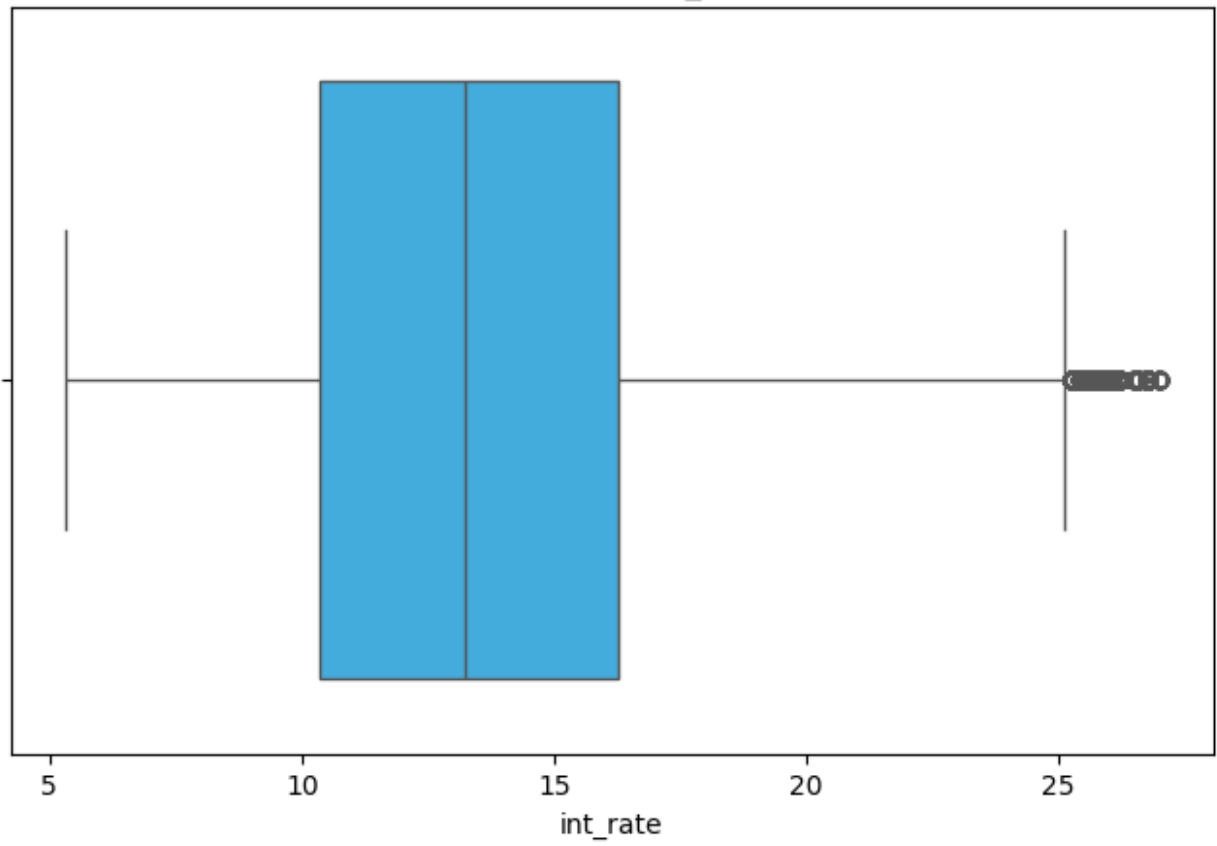
def box_plot(col):
    if col in df.columns:
        plt.figure(figsize=(8, 5))
        sns.boxplot(x=df[col], color="#29B6F6")
        plt.title('Boxplot for {}'.format(col))
        plt.show()
    else:
        print(f"Column '{col}' not found in the DataFrame.")

for col in num_cols:
    box_plot(col)
```

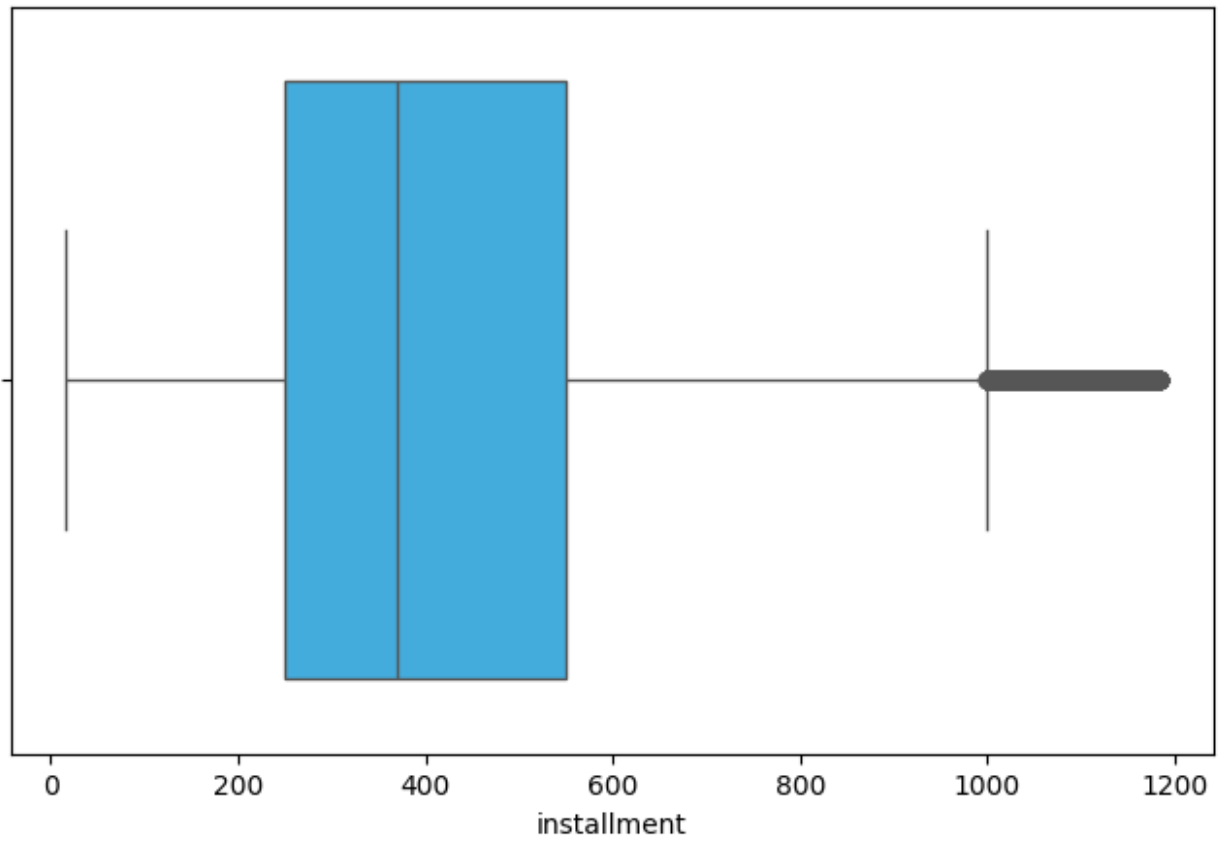
Boxplot for loan_amnt



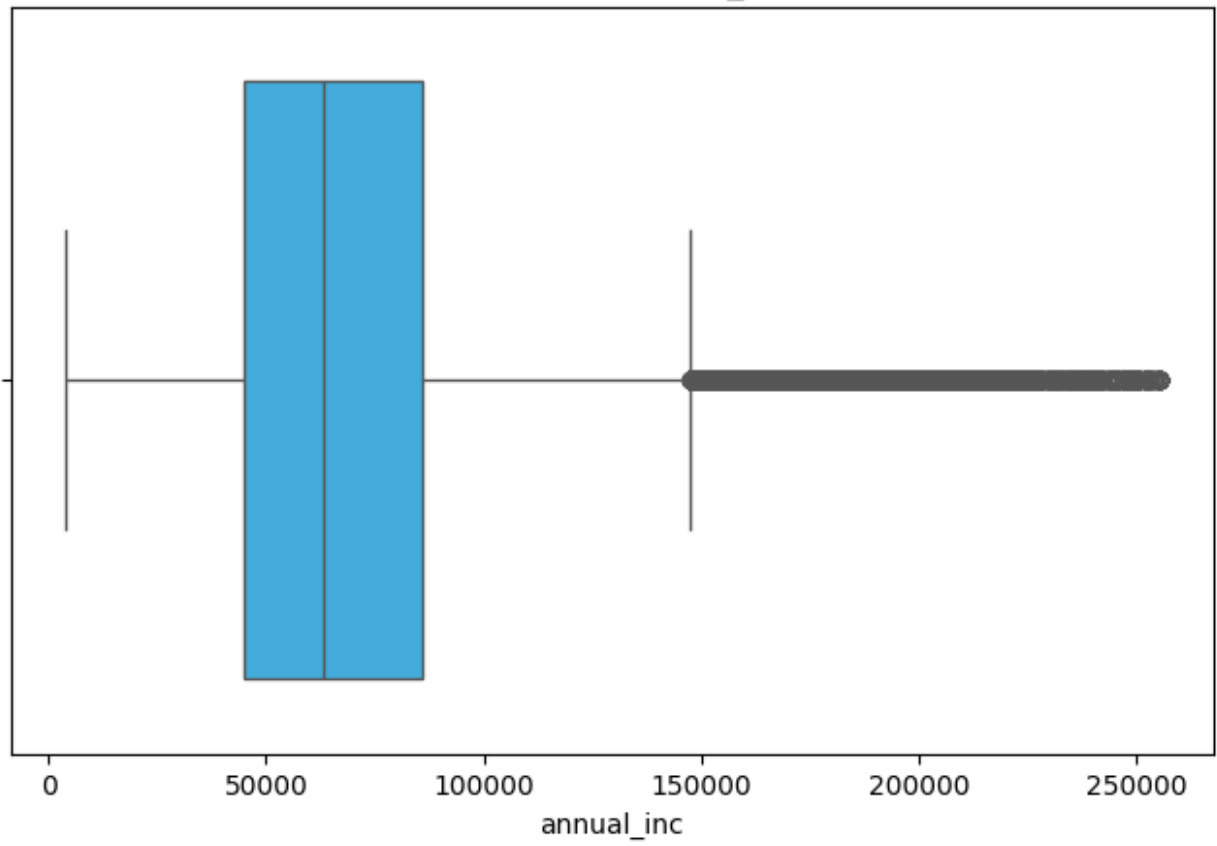
Boxplot for int_rate



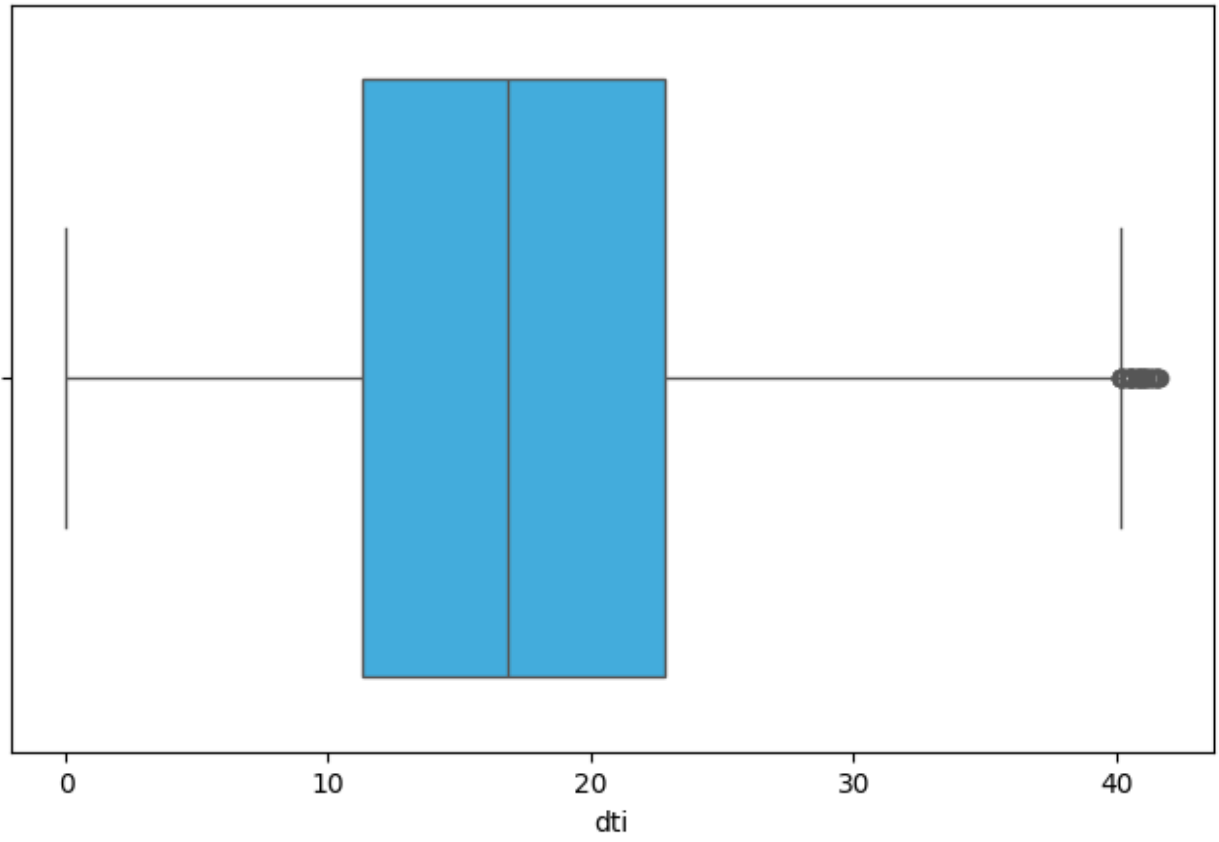
Boxplot for installment



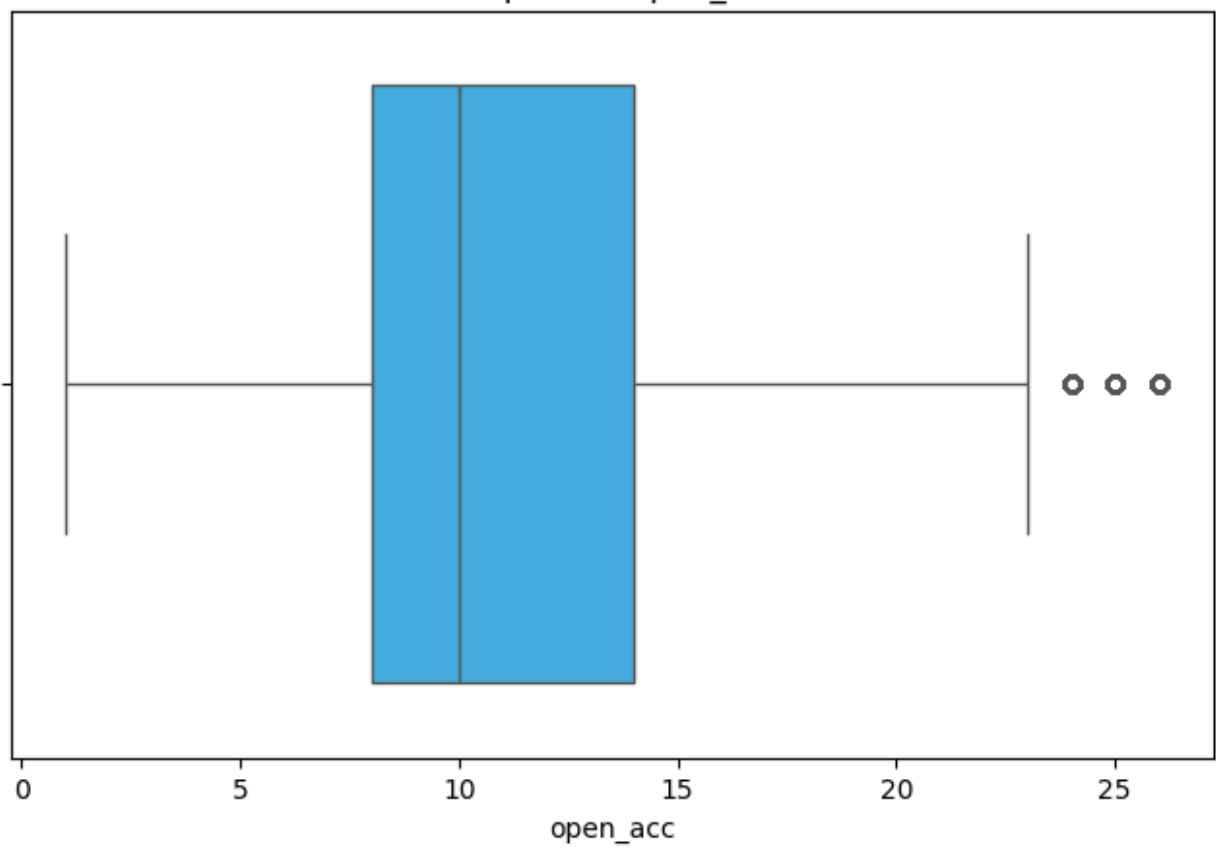
Boxplot for annual_inc



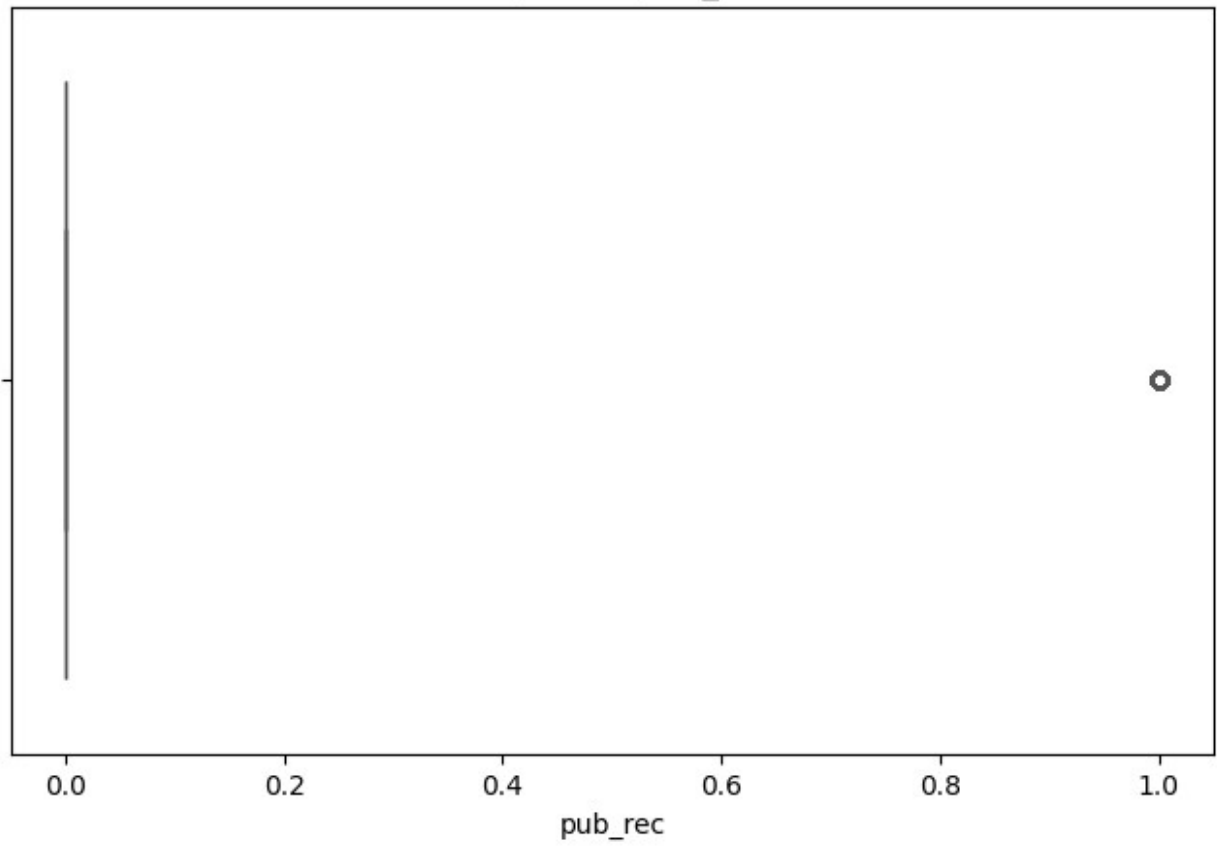
Boxplot for dti



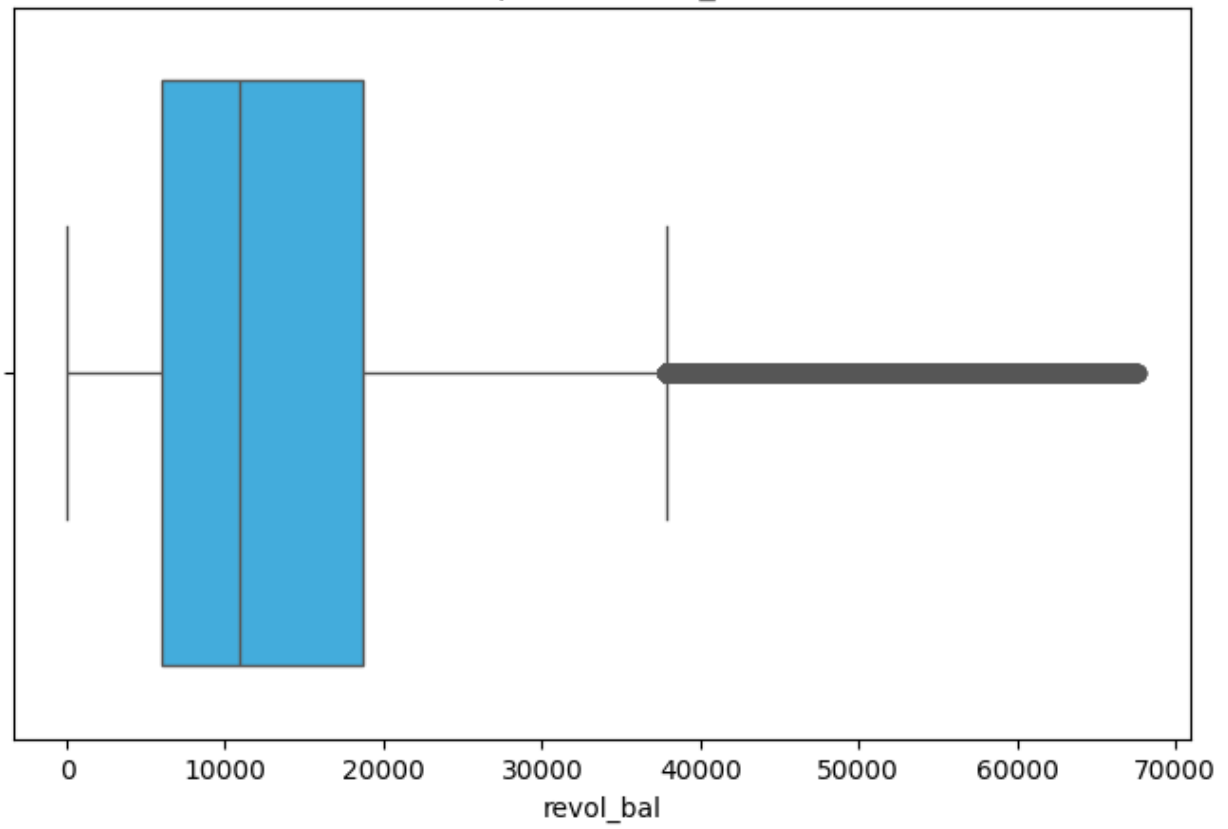
Boxplot for open_acc



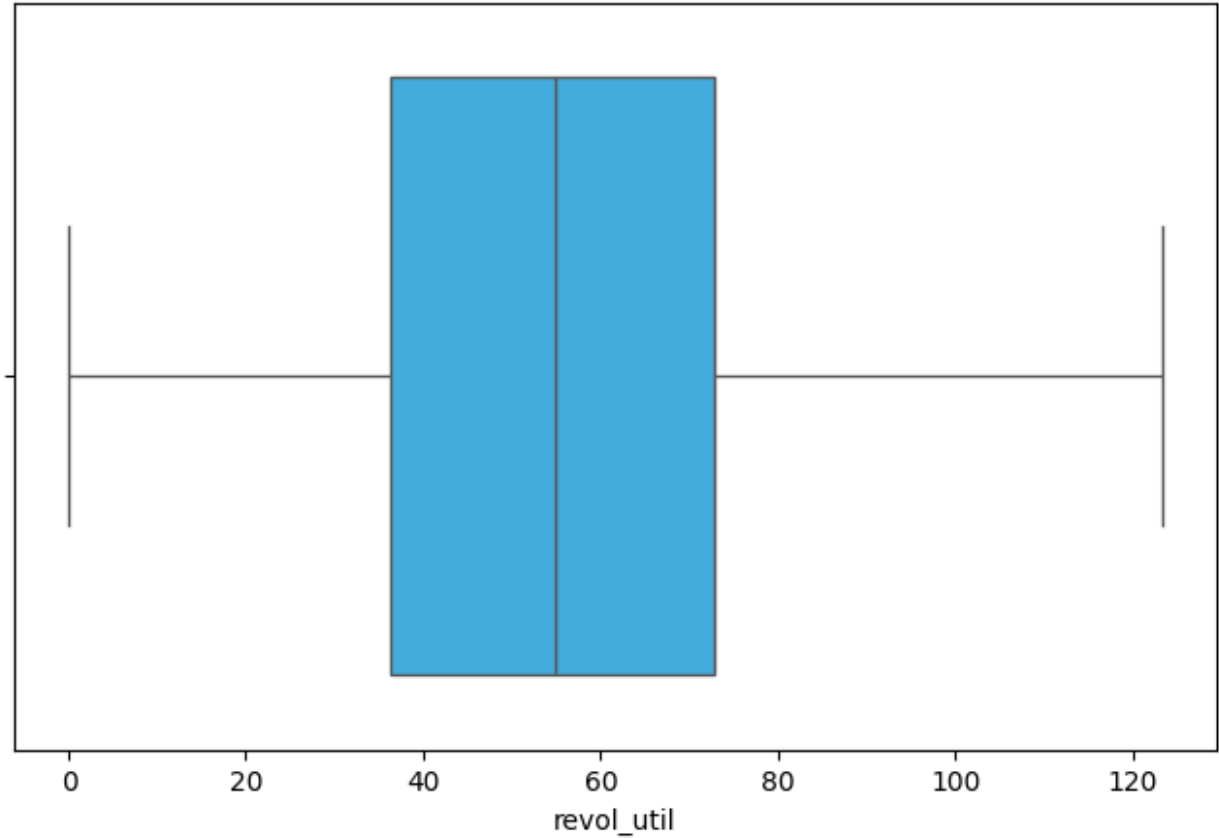
Boxplot for pub_rec



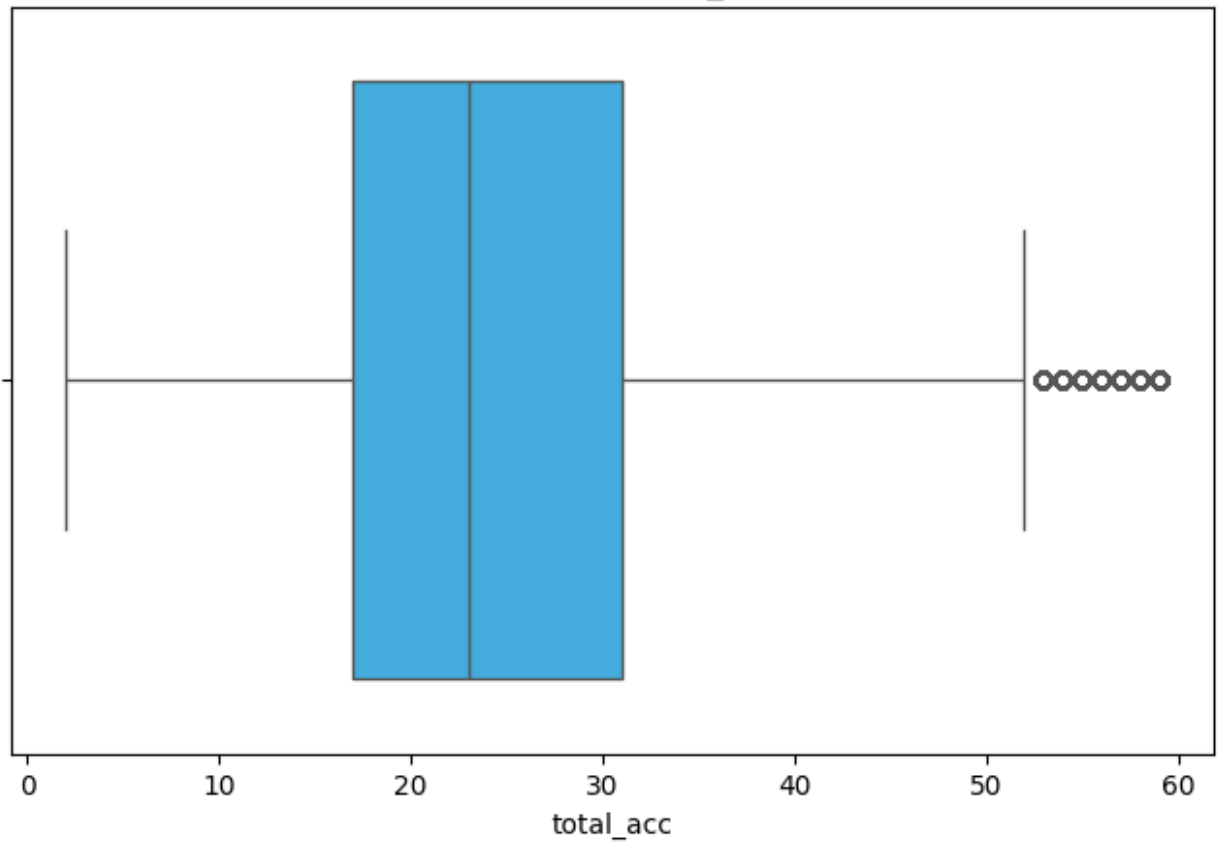
Boxplot for revol_bal



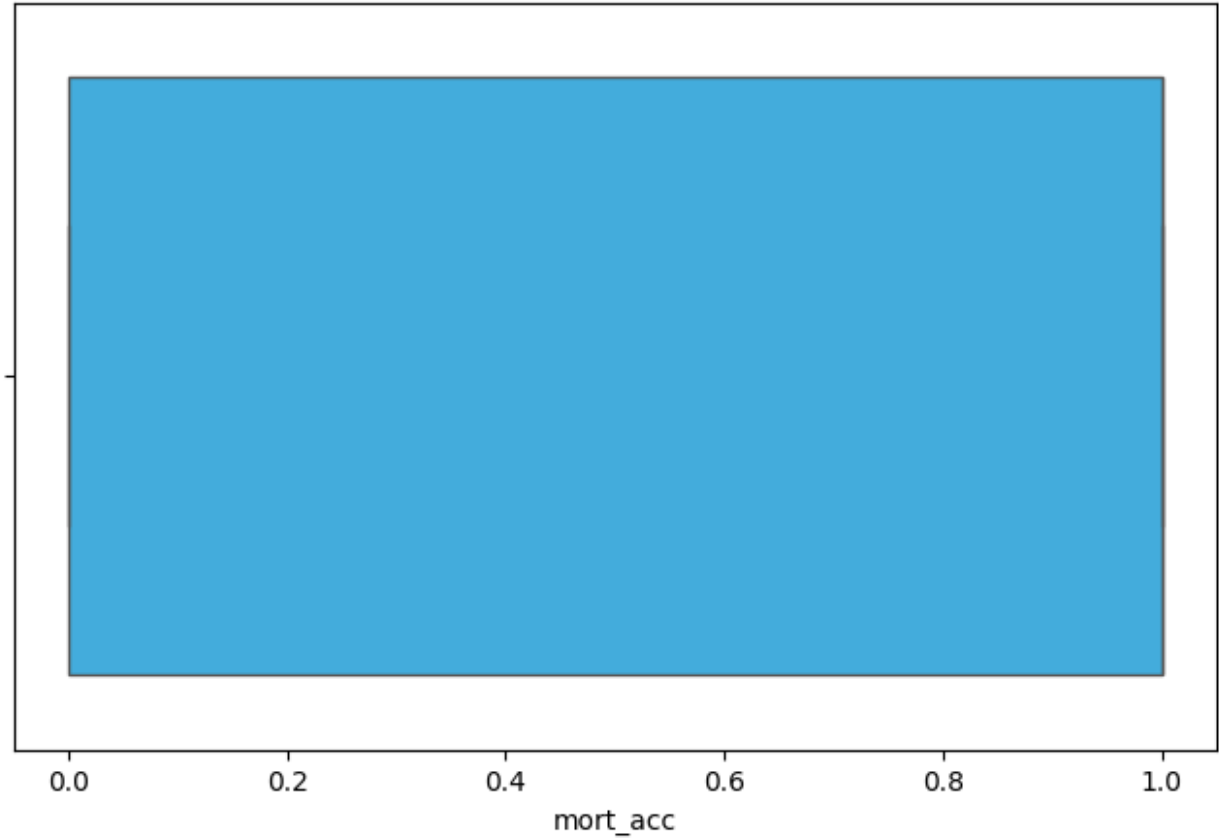
Boxplot for revol_util



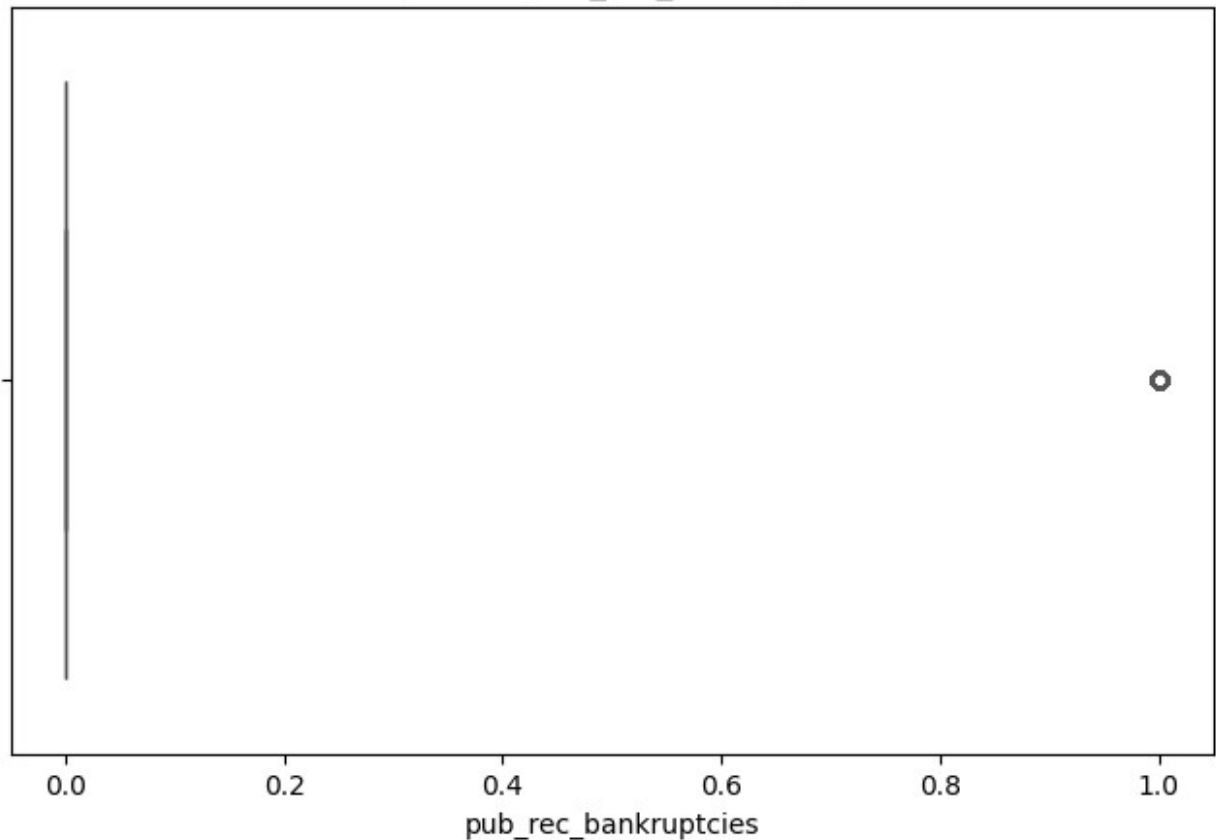
Boxplot for total_acc



Boxplot for mort_acc



Boxplot for pub_rec_bankruptcies



```
term_values = {' 36 months': 36, ' 60 months': 60}
df['term'] = df['term'].map(term_values)

df['loan_status'] = df['loan_status'].map({'Fully Paid': 0, 'Charged
Off': 1})

list_status = {'w': 0, 'f': 1}

df['initial_list_status'] = df['initial_list_status'].map(list_status)

df['zip_code'] = df['address'].apply(lambda x: x[-5:])

df['zip_code'].value_counts(normalize=True) * 100
```

zip_code	
70466	14.375337
30723	14.289710
22690	14.272299
48052	14.127019
00813	11.605591
29597	11.548792
05113	11.519108

```

93700      2.768605
11650      2.762896
86630      2.730643
Name: proportion, dtype: float64

# Dropping of unnecessary columns

unnecessary_columns=['issue_d', 'emp_title', 'title',
                    'sub_grade', 'address', 'earliest_cr_line', 'emp_length']

df.drop(unnecessary_columns,axis=1, inplace=True)

```

one hot encoding

```

dummies=['purpose', 'zip_code', 'grade', 'verification_status',
         'application_type', 'home_ownership']

data=pd.get_dummies(df,columns=dummies,drop_first=True)

pd.set_option('display.max_columns',None)

pd.set_option('display.max_rows',None)

```

Slit data into training and testing

```

from sklearn.model_selection import train_test_split

X=data.drop('loan_status',axis=1)
y=data['loan_status']
X_train, X_test, y_train, y_test
=train_test_split(X,y,test_size=0.30,stratify=y,random_state=42)
print(X_train.shape)
print(X_test.shape)

(245249, 51)
(105108, 51)

```

Model building

```

# Importing stats libraries

from sklearn.linear_model import LogisticRegression
from sklearn import metrics
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
from sklearn.metrics import roc_auc_score
from sklearn.metrics import roc_curve
from sklearn.metrics import precision_recall_curve

```

```

from sklearn.model_selection import train_test_split, KFold,
cross_val_score
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report,
    roc_auc_score, roc_curve, auc,
    ConfusionMatrixDisplay, RocCurveDisplay
)
from statsmodels.stats.outliers_influence import
variance_inflation_factor

```

Scaling

```

scaler = MinMaxScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

logreg=LogisticRegression(max_iter=1000)
logreg.fit(X_train,y_train)
LogisticRegression(max_iter=1000)
y_pred = logreg.predict(X_test)

print('Accuracy of Logistic Regression Classifier on test set:
{:.3f}'.format(logreg.score(X_test, y_test)))
Accuracy of Logistic Regression Classifier on test set: 0.891
print(classification_report(y_test,y_pred))

```

	precision	recall	f1-score	support
0	0.89	0.99	0.94	84915
1	0.94	0.46	0.62	20193
accuracy			0.89	105108
macro avg	0.91	0.73	0.78	105108
weighted avg	0.90	0.89	0.88	105108

```

#Plot confusion Matrix

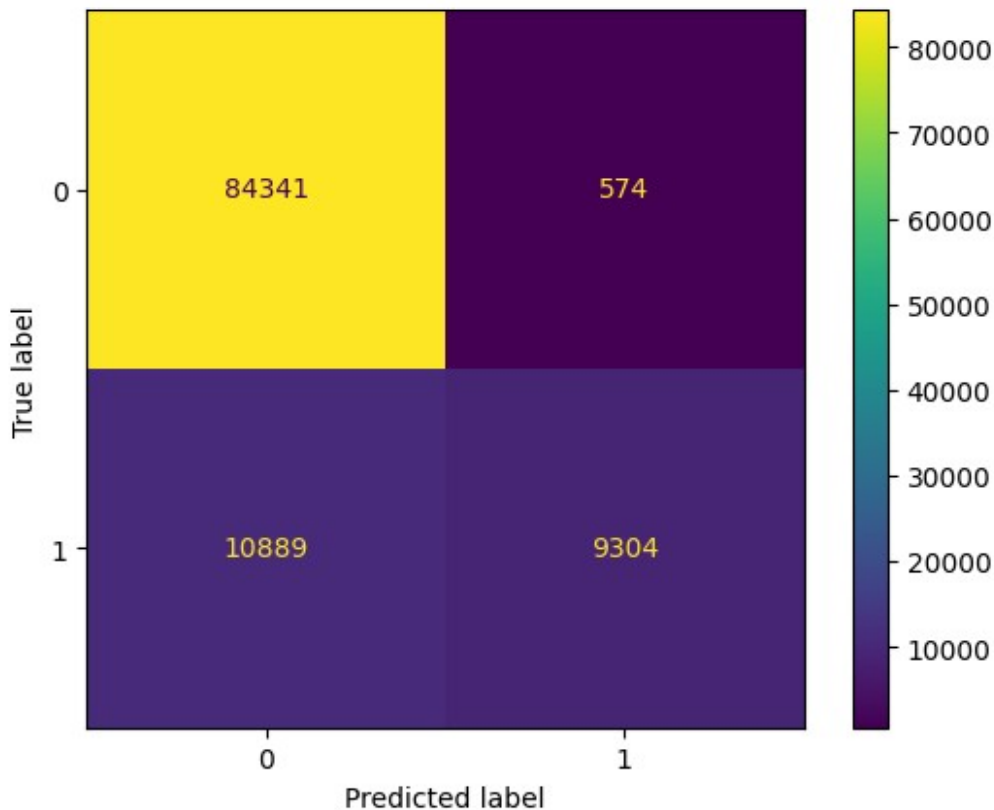
confusion_matrix=confusion_matrix(y_test,y_pred)
print(confusion_matrix)

ConfusionMatrixDisplay(confusion_matrix=confusion_matrix,
display_labels=logreg.classes_).plot()

```

```
[[84341  574]
 [10889 9304]]
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at
0x1889a9f9e50>
```



ROC Curve -

An ROC curve (receiver operating characteristic curve) is a graph showing the performance of a classification model at all classification thresholds. This curve plots two parameters:

- True Positive Rate
- False Positive Rate

True Positive Rate (TPR) is a synonym for recall and is therefore defined as follows:

- $TPR = \frac{TP}{TP + FN}$

False Positive Rate (FPR) is defined as follows:

- $FPR = \frac{FP}{FP + TN}$

An ROC curve plots TPR vs. FPR at different classification thresholds. Lowering the classification threshold classifies more items as positive, thus increasing both False Positives and True Positives. The following figure shows a typical ROC curve.

AUC (Area under the ROC Curve) -

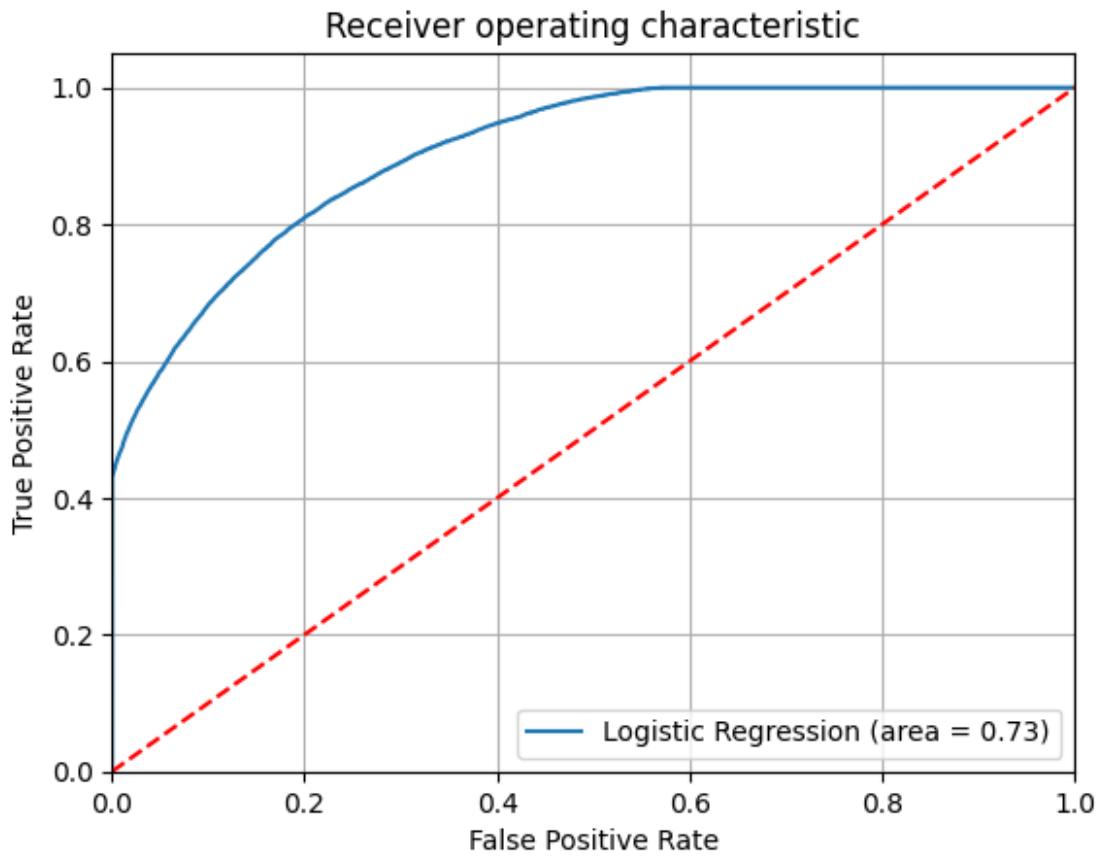
AUC stands for "Area under the ROC Curve." That is, AUC measures the entire two-dimensional area underneath the entire ROC curve (think integral calculus) from (0,0) to (1,1).

AUC provides an aggregate measure of performance across all possible classification thresholds. One way of interpreting AUC is as the probability that the model ranks a random positive example more highly than a random negative example. For example, given the following examples, which are arranged from left to right in ascending order of logistic regression predictions:

```
logit_roc_auc = roc_auc_score(y_test, logreg.predict(X_test))

fpr, tpr, thresholds = roc_curve(y_test, logreg.predict_proba(X_test)[:,1])

plt.figure()
plt.plot(fpr, tpr, label='Logistic Regression (area = %0.2f)' %
logit_roc_auc)
plt.plot([0, 1], [0, 1], 'r--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver operating characteristic')
plt.legend(loc="lower right")
plt.grid()
plt.show()
```



Insights:

- ROC-AUC curve is crossing the area near about 0.73 which indicates that model is performing well.
- There is still room for some model improvement
- By collecting more data, using a more complex model, or tuning the hyperparameters, it is possible to improve the model's performance.

Precision-Recall Curve

```
precisions, recalls, thresholds = precision_recall_curve(y_test,
logreg.predict_proba(X_test)[:, 1])

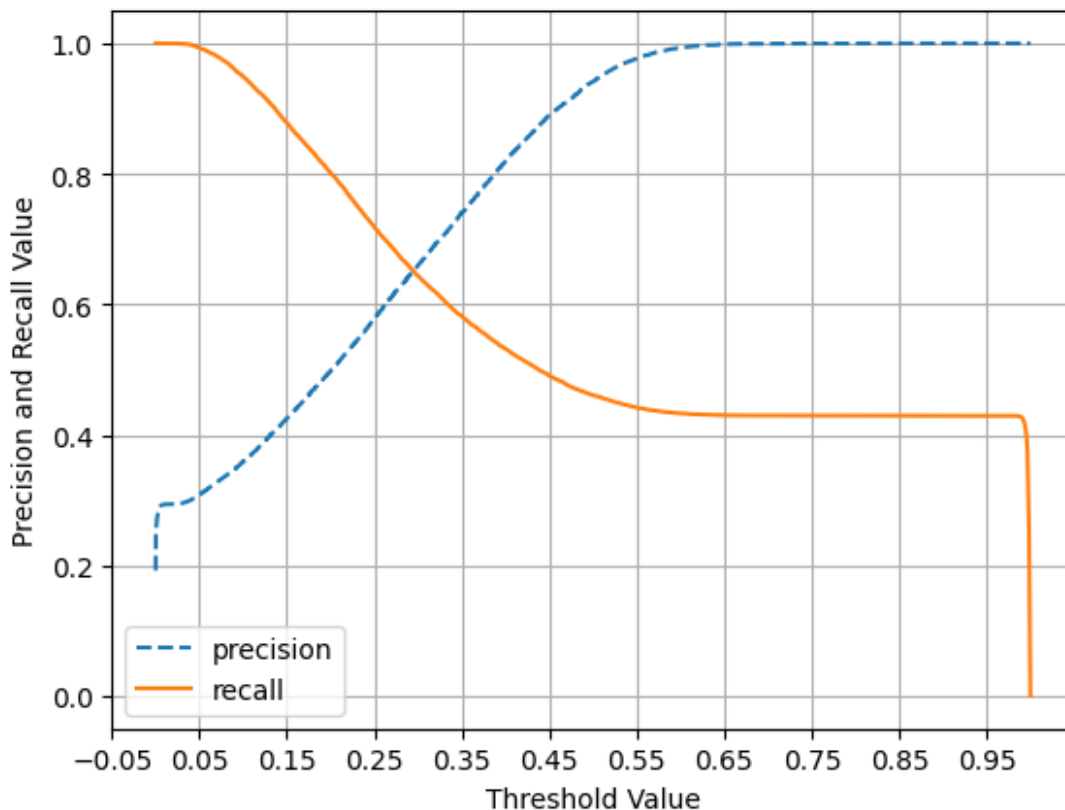
threshold_boundary = thresholds.shape[0]

# Plot precision
plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--',
label='precision')

# Plot recall
plt.plot(thresholds, recalls[0:threshold_boundary], label='recall')

start, end = plt.xlim()
plt.xticks(np.round(np.arange(start, end, 0.1), 2))
```

```
plt.xlabel('Threshold Value')
plt.ylabel('Precision and Recall Value')
plt.legend()
plt.grid()
plt.show()
```



Insights

- Precision score is highest at 0.55 threshold. High precision value indicates that model is positively predicating the charged off loan status which helps business to take more stable decision.
- Recall score is higher on smaller threshold but after 0.55 the recall value is constant. Model is correctly classifying the actual predicated values as instances.

Actional Insights and Recommendations

1. 80% of the customers have paid the loan fully.
2. 20% of the customers are the defaulters.
3. The organization can the trained model to make prediction for whether a person will likely to pay the loan amount or he will be a defaulter.
4. Model achieves the 94% f1-score for the negative class (Fully Paid).
5. Model achieves the 62% f1-score for the positive class (Charged off).

6. Cross Validation accuracy and testing accuracy is almost same which infers model is performing the decent job. We can trust this model for unseen data
7. By collecting more data, using a more complex model, or tuning the hyperparameters, it is possible to improve the model's performance.
8. ROC AUC curve area of 0.73, the model is correctly classifying about 73% of the instances. This is a good performance, but there is still room for improvement.
9. The precision-recall curve allows us to see how the precision and recall trade-off as we vary the threshold. A higher threshold will result in higher precision, but lower recall, and vice versa. The ideal point on the curve is the one that best meets the needs of the specific application.
10. After balancing the dataset, there is significant change observed in the precision and recall score for both of the classes.
11. Accuracy of Logistic Regression Classifier on test set: 0.891 which is decent and not by chance